



A ■ P ■ U

**ASIA PACIFIC UNIVERSITY
OF TECHNOLOGY & INNOVATION**

NAME	TP NUM
SERENE LOH ZI TING	TP075920
SIEW ZHEN LYNN	TP076386
YONG XUAN LYN	TP076797

Contents

Introduction.....	3
Assumptions.....	4
Design of the program	5
Program source code.....	67
Additional features source code.....	116
Screenshots of sample input/output with explanation	119
Conclusion	122
Task Division	123

Introduction

Kuala Lumpur, KL Central Computer Company (KLCCC) is a quickly growing company that sells and repairs computers. KLCCC has made the decision to use an automated Python software solution in order to streamline its increasing volume of transactions. Three main functional areas which is inventory management, user management, and customer management, are managed by specialised developers within this system. To ensure effective management and oversight of all business transactions, the system will support functionalities for user registration, login, and various operational tasks based on user roles Customer, Admin, and Super User.

Assumptions

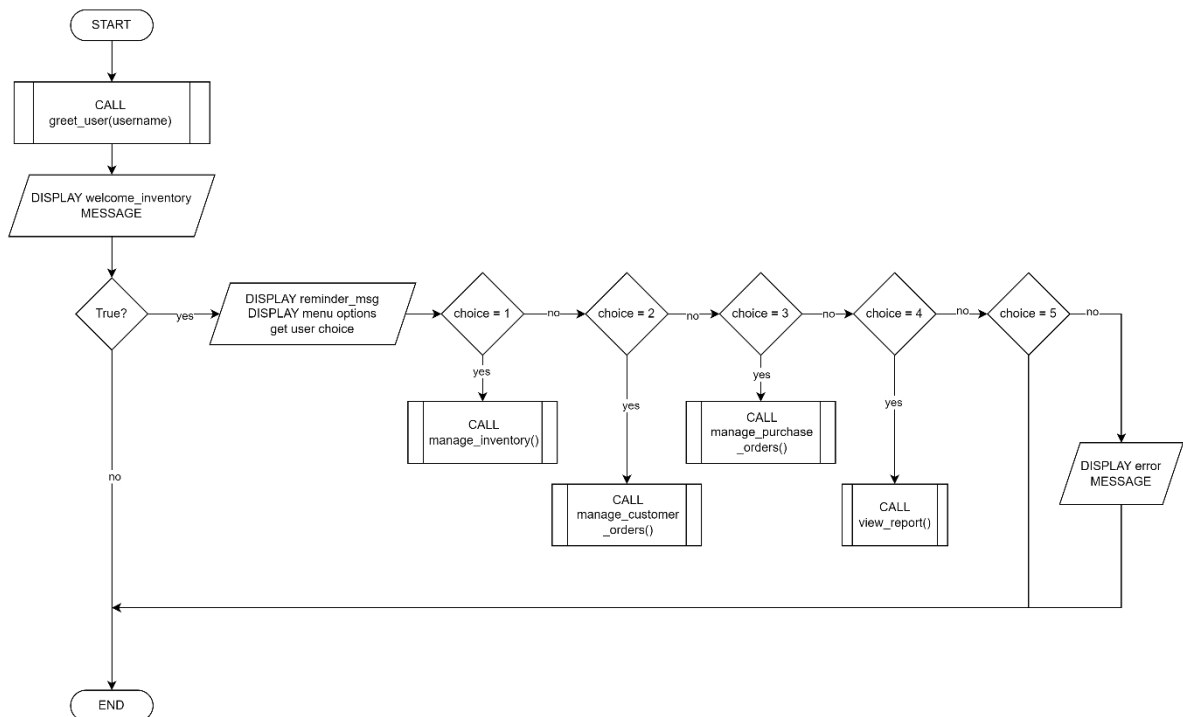
1. Customer Password must total ≥ 8 characters, including 1 uppercase letter **and** 3 integers.
2. Password for inventory must total ≥ 8 characters, starting with INV@ and followed by numbers.
3. Password for admin must be total of ≥ 10 characters, starting with ADMIN@.
4. The username for the superuser is “superuser@123” and password for the superuser is “superusernotsuperman”
5. The username for admin is auto generated and it’s unique.
6. There are two (2) types of purchase orders:
 - a. Prepared and sent by buyers to place orders
 - b. Prepared and sent by seller to supplier to purchase stock
7. There will only be two (2) inventory staff, who will be identified directly by their names.
8. No tax is applied to the item purchased/sold.
9. Each supplier purchase order can only order 1 item.
10. The duration for all services/ repair is three (3) days only.

Design of the program

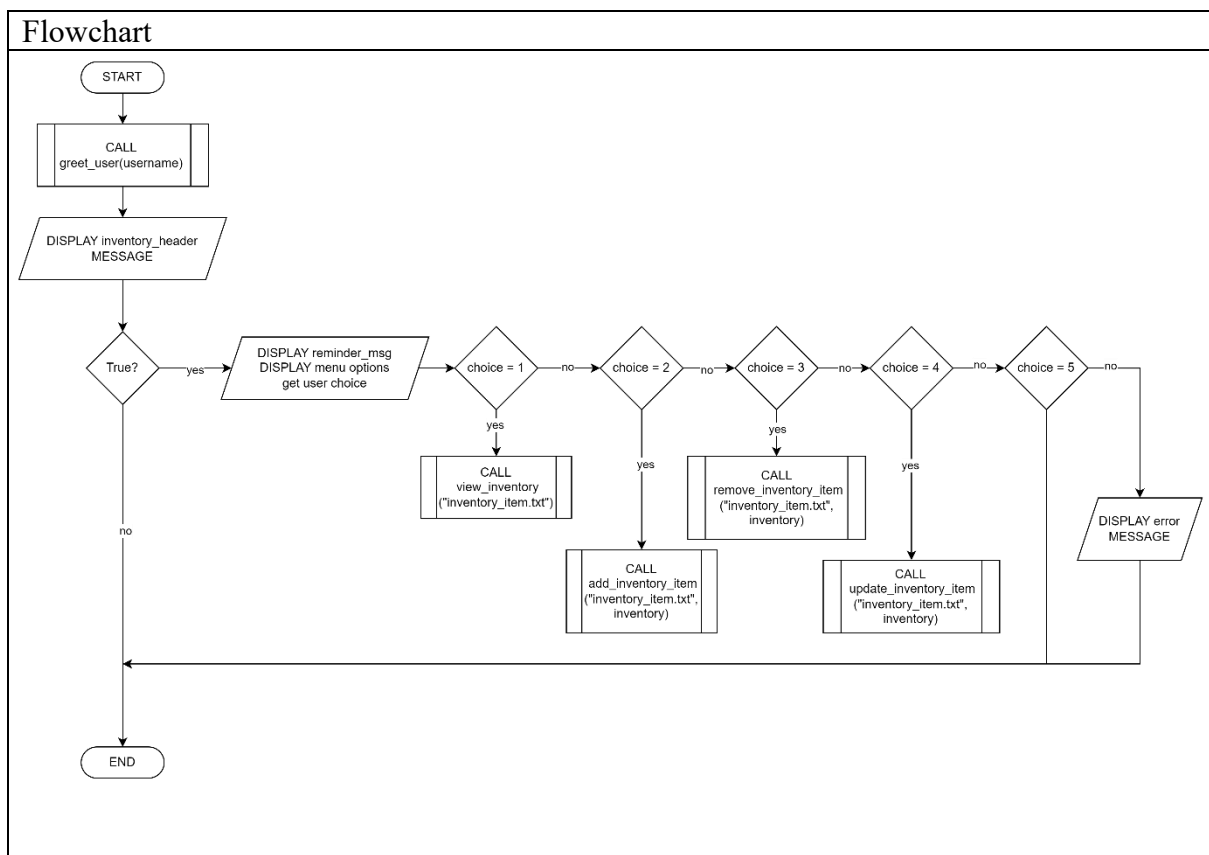
using pseudocode and flowcharts – which adheres to the requirements provided above

Function	Inventory_Main_Menu
Pseudocode	<pre>Inventory_Main_Menu START CALL greet_user(username) DISPLAY welcome_inventory MESSAGE WHILE True DISPLAY reminder_msg DISPLAY menu options INPUT choice IF choice IS EQUAL TO 1 CALL manage_inventory() ELIF choice IS EQUAL TO 2 CALL manage_customer_orders() ELIF choice IS EQUAL TO 3 CALL manage_purchase_orders() ELIF choice IS EQUAL TO 4 CALL view_report() ELIF choice IS EQUAL TO 5 BREAK ELSE DISPLAY error MESSAGE ENDIF ENDWHILE END</pre>

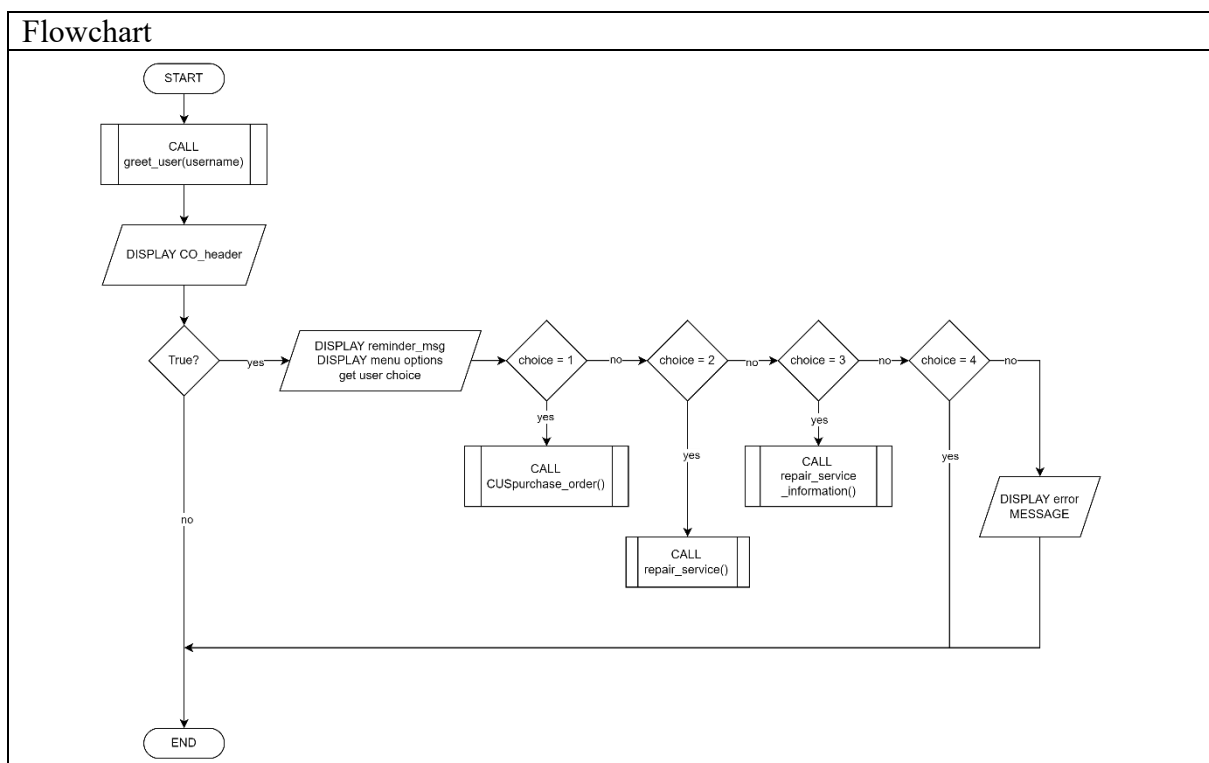
Flowchart



Function	Manage_Inventory_Item
Pseudocode	<pre> Manage_Inventory_Item START CALL greet_user(username) DISPLAY inventory_header MESSAGE WHILE True DISPLAY reminder_msg DISPLAY menu options INPUT choice IF choice IS EQUAL TO 1 CALL view_inventory("inventory_item.txt") ELIF choice IS EQUAL TO 2 CALL add_inventory_item("inventory_item.txt", inventory) ELIF choice IS EQUAL TO 3 CALL remove_inventory_item("inventory_item.txt", inventory) ELIF choice IS EQUAL TO 4 CALL update_inventory_item("inventory_item.txt", inventory) ELIF choice IS EQUAL TO 5 BREAK ELSE DISPLAY error MESSAGE ENDIF ENDWHILE END </pre>



Function	Manage_Customer_Orders
Pseudocode	<pre> Manage_Customer_Orders START CALL greet_user(username) PRINT CO_header WHILE TRUE PRINT reminder_msg PRINT "\n(Example: enter '1' to view customer purchase order.)" PRINT "\nPlease select your action:\n" PRINT "1. View Customer Purchase Order\n" PRINT "2. View Repair/Service Order\n" PRINT "3. Repair/Service Information\n" PRINT "4. Return to Inventory Main Menu\n" SET choice TO INPUT("Your input: ") IF choice IS NUMERIC CONVERT choice TO INTEGER IF choice EQUALS 1 CALL CUSpurchase_order() ELSEIF choice EQUALS 2 CALL repair_service() ELSEIF choice EQUALS 3 CALL repair_service_information() ELSEIF choice EQUALS 4 BREAK WHILE ELSE PRINT "Enter selections displayed only." ELSE PRINT "Enter option NUMBER only." ENDWHILE END </pre>



Function: Manage Purchase Order

Pseudocode

```
Manage_Purchase_Order

START
    CALL `greet_user(username)`

    DISPLAY `PO_header`

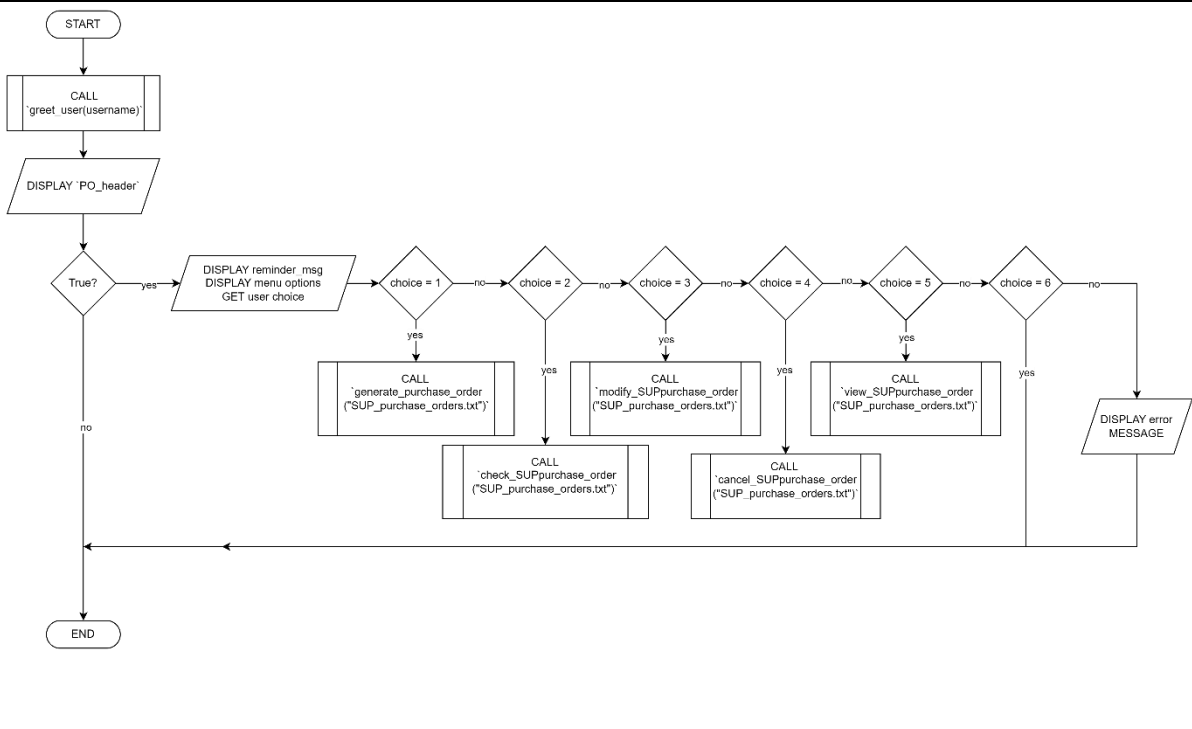
    WHILE True
        DISPLAY `reminder_msg`
        DISPLAY OPTIONS:
            "1. Generate Purchase Order"
            "2. Check Supplier Purchase Order Status"
            "3. Modify Purchase Order"
            "4. Cancel Supplier Purchase Order"
            "5. View Supplier Purchase Order"
            "6. Return to Inventory Main Menu"

        PROMPT USER FOR CHOICE AND CONVERT TO INTEGER AS `choice`

        IF `choice` EQUALS 1
            CALL `generate_purchase_order("SUP_purchase_orders.txt")`
        ELIF `choice` EQUALS 2
            CALL `check_SUPpurchase_order("SUP_purchase_orders.txt")`
        ELIF `choice` EQUALS 3
            CALL `modify_SUPpurchase_order("SUP_purchase_orders.txt")`
        ELIF `choice` EQUALS 4
            CALL `cancel_SUPpurchase_order("SUP_purchase_orders.txt")`
        ELIF `choice` EQUALS 5
            CALL `view_SUPpurchase_order("SUP_purchase_orders.txt")`
        ELIF `choice` EQUALS 6
            BREAK
        ELSE
            DISPLAY "Enter selections displayed only."
        ENDIF

        IF `ValueError` OCCURS
            DISPLAY "Enter option NUMBER only."
        ENDIF
    ENDWHILE
END
```

Flowchart



Function	View_Reports
Pseudocode	<pre> View_Reports START CALL greet_user(username) DISPLAY view_report_header MESSAGE WHILE True DISPLAY reminder_msg DISPLAY menu options INPUT choice IF choice IS EQUAL TO 1 CALL invoice_report() ELIF choice IS EQUAL TO 2 CALL inventory_report() ELIF choice IS EQUAL TO 3 BREAK ELSE DISPLAY error MESSAGE ENDIF ENDWHILE END </pre>
Flowchart	<pre> graph TD START([START]) --> Greet[CALL greet_user(username)] Greet --> DisplayHeader[/DISPLAY view_report_header MESSAGE/] DisplayHeader --> True{True?} True -- yes --> DisplayMenu[/DISPLAY reminder_msg DISPLAY menu options GET user choice/] DisplayMenu --> Choice1{choice = 1} Choice1 -- yes --> CallInvoice[CALL invoice_report()] Choice1 -- no --> Choice2{choice = 2} Choice2 -- yes --> CallInventory[CALL inventory_report()] Choice2 -- no --> Choice3{choice = 3} Choice3 -- yes --> DisplayError[/DISPLAY error MESSAGE/] Choice3 -- no --> DisplayError DisplayError --> End([END]) True -- no --> End </pre>

Function	View_Inventory
Pseudocode	<pre> View_Inventory START OPEN "inventory_item.txt" FOR READING PRINT THE CONTENTS OF THE FILE END </pre>
Flowchart	<pre> graph TD Start([START]) --> Process[OPEN "inventory_item.txt" FOR READING] Process --> Output[/PRINT THE CONTENTS OF THE FILE/] Output --> End([END]) </pre>

Function: Add Inventory Item

Pseudocode

Add Inventory Item

START

 INITIALIZE `existing\_item\_ids` AS a set containing the first element (ItemID) of each item in `inventory\_list`

 CALL `get\_next\_item\_id` WITH `existing\_item\_ids` AND STORE RESULT IN `item\_id`

 CALL `get\_next\_sku` WITH `item\_id` AND STORE RESULT IN `sku`

 PROMPT USER FOR `item\_name` AND STORE INPUT

 PROMPT USER FOR `brand` AND STORE INPUT

 PROMPT USER FOR `category` AND STORE INPUT

 PROMPT USER FOR `model\_number` AND STORE INPUT

 PROMPT USER FOR `description` AND STORE INPUT

 PROMPT USER FOR `specification` AND STORE INPUT

 CALL `get\_valid\_int` WITH "Enter Warranty (Year): " AND STORE RESULT IN `warranty`

 CALL `get\_valid\_float` WITH "Enter Price/Unit (RM): " AND STORE RESULT IN `price\_per\_unit`

 CALL `get\_valid\_int` WITH "Enter Quantity In Stock: " AND STORE RESULT IN `quantity\_in\_stock`

 CREATE `new\_item` LIST WITH ALL THE ABOVE VARIABLES

 APPEND `new\_item` TO `inventory\_list`

 OPEN `"inventory\_item.txt"` IN APPEND MODE

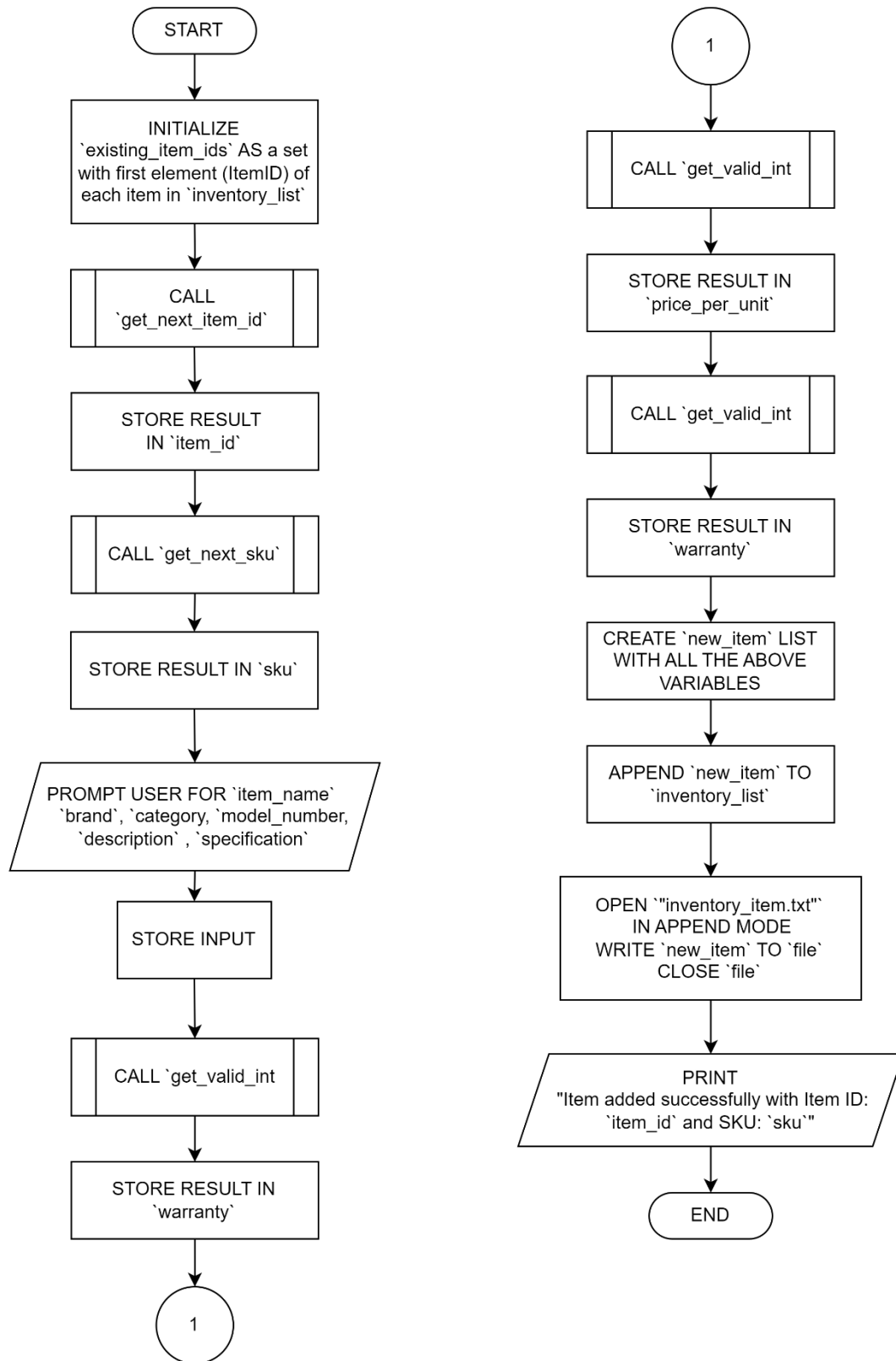
 | WRITE `new\_item` TO `file`

 CLOSE `file`

 PRINT "Item added successfully with Item ID: `item\_id` and SKU: `sku`"

END

Flowchart



Function: Remove Inventory Item

Pseudocode

Remove Inventory Item

START

PROMPT USER TO ENTER `item\_id` (Item ID of the item to remove) AND STORE INPUT

INITIALIZE `item\_to\_remove` AS `None`

FOR EACH `item` IN `inventory\_list`

IF `item[0]` (Item ID) EQUALS `item\_id`

SET `item\_to\_remove` AS `item`

BREAK

ENDIF

ENDFOR

IF `item\_to\_remove` IS NOT `None`

PRINT "Item found: `item\_to\_remove`"

PROMPT USER FOR CONFIRMATION ("yes/no") AND STORE INPUT AS `confirmation`

IF `confirmation` EQUALS "yes"

REMOVE `item\_to\_remove` FROM `inventory\_list`

OPEN `"inventory\_item.txt"` IN WRITE MODE

FOR EACH `item` IN `inventory\_list`

WRITE `item` TO `file`

ENDFOR

CLOSE `file`

PRINT "Item with ID `item\_id` has been removed from the inventory."

ELSE

PRINT "No changes were made."

ENDIF

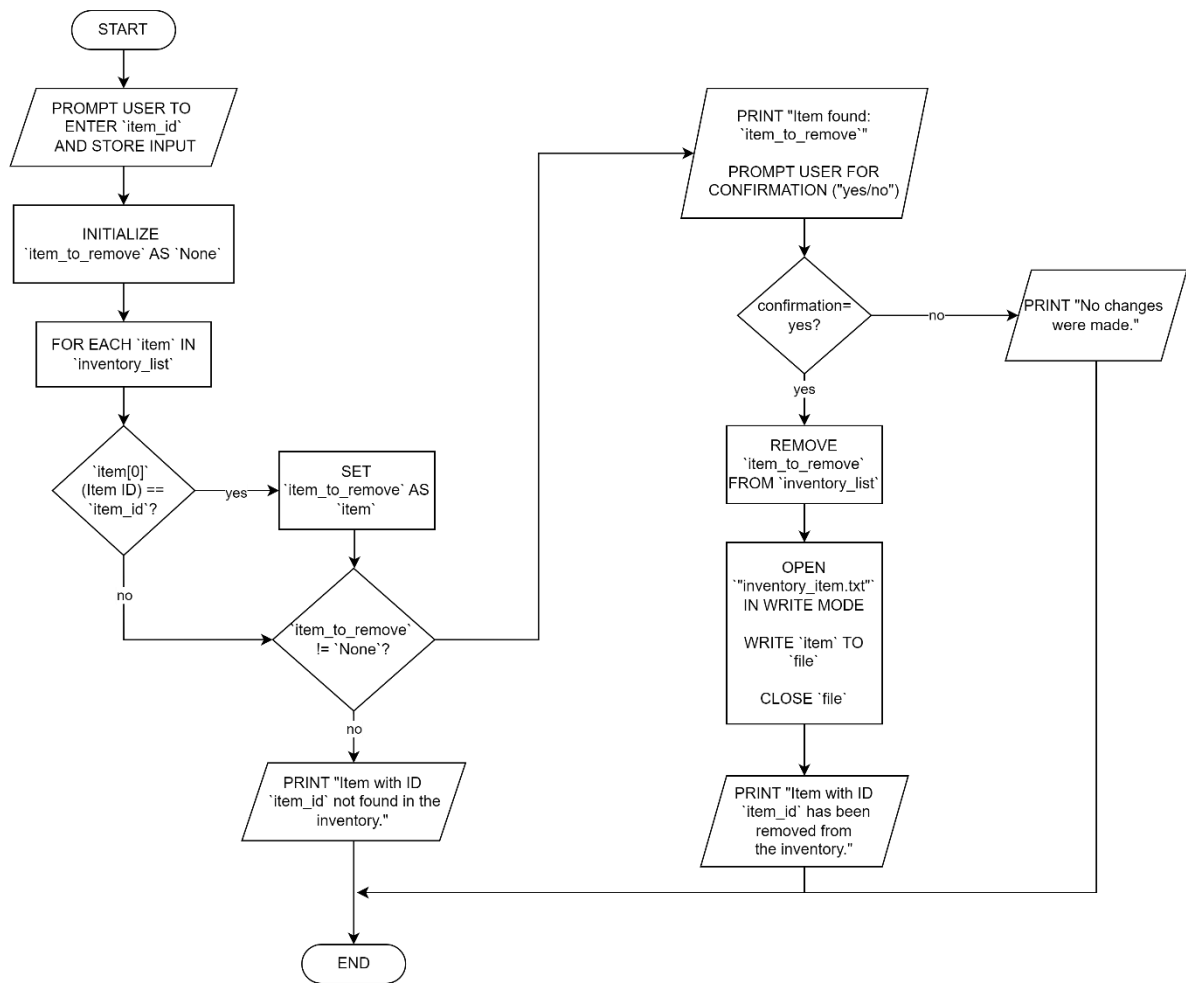
ELSE

PRINT "Item with ID `item\_id` not found in the inventory."

ENDIF

END

Flowchart



Function: Update Inventory Item

Pseudocode

```
Update Inventory Item

START
    SET item_id TO input from user (trimmed)

    SET item_to_update TO None

    FOR each item IN inventory_list
        IF item[0] EQUALS item_id THEN
            SET item_to_update TO item
            BREAK loop
        ENDIF
    ENDFOR

    IF item_to_update EXISTS THEN
        PRINT item details

        SET attributes TO list of attributes to be updated

        PRINT options for attributes with their corresponding indices

        SET choice TO get valid integer input from user

        IF choice IS between 1 and length of attributes THEN
            SET attribute_name TO attributes[choice - 1]

            SET mapping TO list of indices corresponding to attributes in item_to_update

            SET correct_index TO mapping[choice - 1]

            SET current_value TO item_to_update[correct_index]

            PRINT current value of selected attribute

            IF choice IS 7 OR 9 THEN
                SET new_value TO get valid integer input from user
            ELSEIF choice IS 8 THEN
                SET new_value TO get valid float input from user
            ELSE
                SET new_value TO input from user (trimmed)
            ENDIF

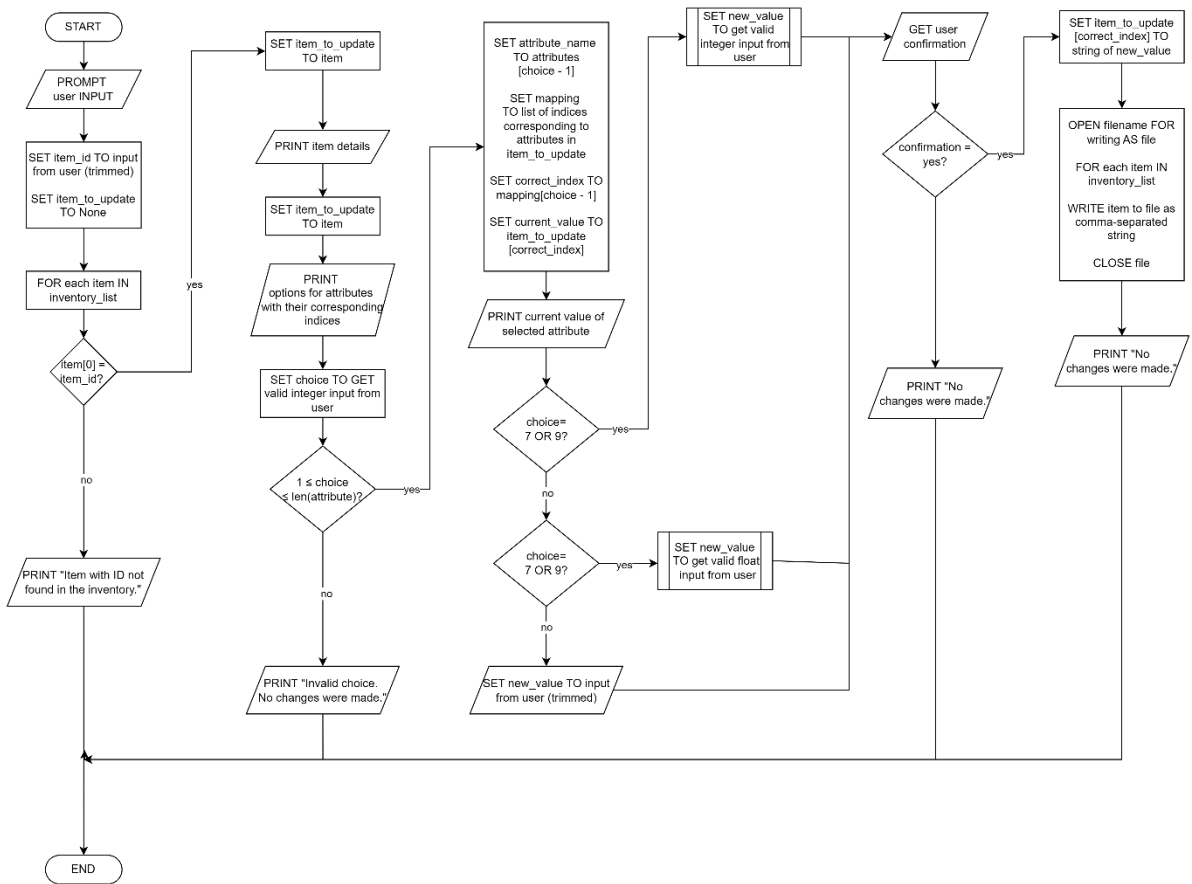
            SET confirmation TO input from user (trimmed and lowercased)

            IF confirmation EQUALS 'yes' THEN
                SET item_to_update[correct_index] TO string of new_value

                OPEN filename FOR writing AS file
                FOR each item IN inventory_list
                    WRITE item to file as comma-separated string
                ENDFOR
                CLOSE file

                PRINT success message
            ELSE
                PRINT "No changes were made."
            ENDIF
        ELSE
            PRINT "Invalid choice. No changes were made."
        ENDIF
    ELSE
        PRINT "Item with ID not found in the inventory."
    ENDIF
END
```

Flowchart



Function: Generate Purchase Order

Pseudocode

```
START
  LOAD existing orders FROM file_path INTO existing_orders

  GET next Order ID USING order_ids FROM existing_orders

  INITIALIZE order WITH order_id AND "1" (for one item per order)

  INITIALIZE total_price TO 0

  WHILE TRUE
    SET item_id TO input from user (trimmed)
    IF item_id EXISTS in inventory THEN
      BREAK loop
    ELSE
      PRINT "Invalid input. Item ID not found in inventory."
    ENDIF
  ENDWHILE

  SET quantity_ordered TO integer input from user

  FIND item in inventory USING item_id
  SET unit_price TO item[9]
  SET price_per_item TO unit_price * quantity_ordered
  ADD price_per_item TO total_price

  APPEND item details TO order (item_id, "Pending", "Unpaid", quantity_ordered, unit_price, price_per_item)

  GET current date and time
  SET order_date TO current date
  SET order_time TO current time

  CALCULATE expected_delivery_date (30 days after order date)

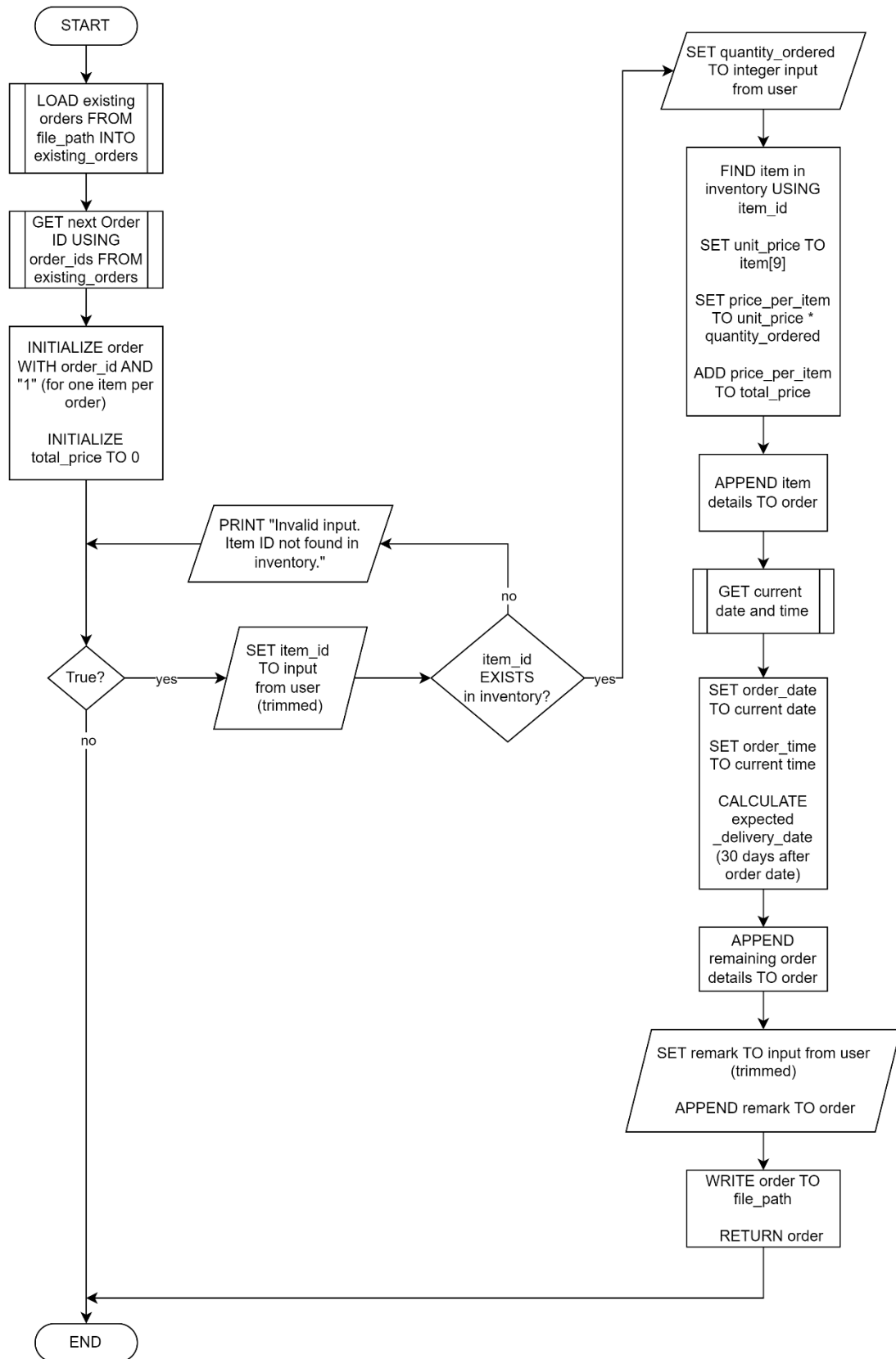
  APPEND remaining order details TO order (order_date, order_time, total_price, expected_delivery_date)

  SET remark TO input from user (trimmed)
  APPEND remark TO order

  WRITE order TO file_path

  RETURN order
END
```

Flowchart



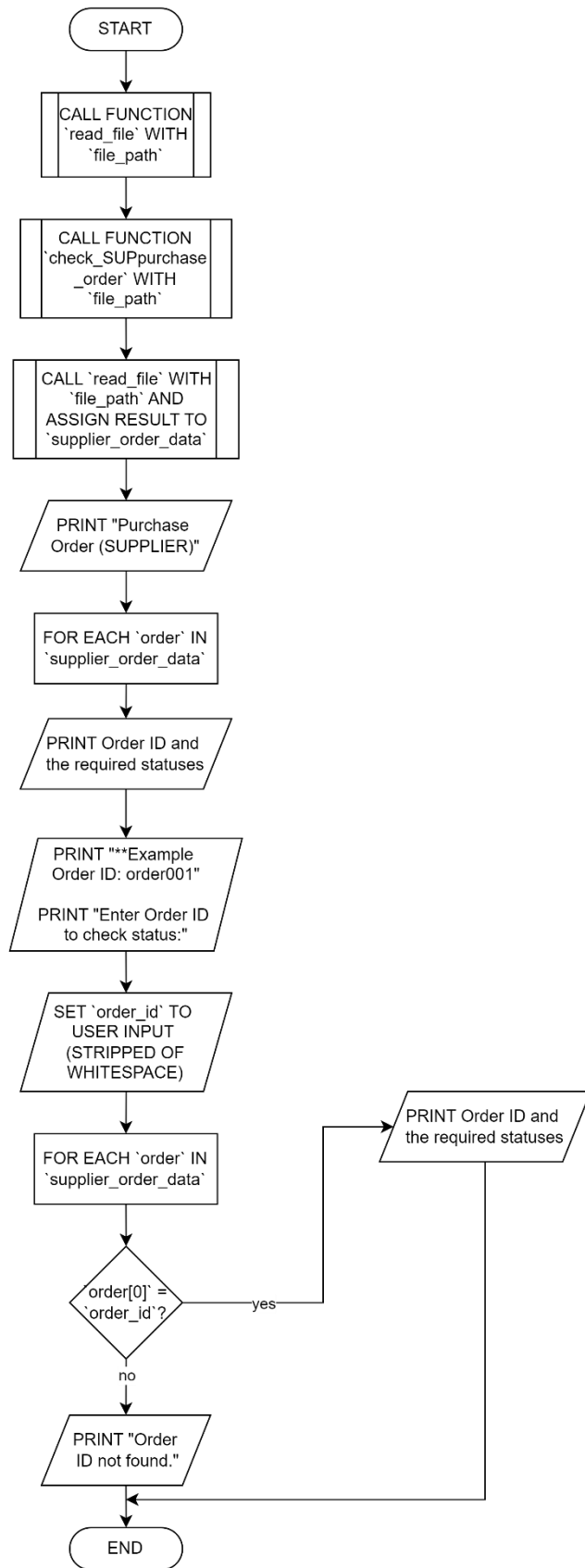
Function: Check Supplier Purchase Order Status

Pseudocode

START

```
CALL FUNCTION `read_file` WITH `file_path`  
    TRY TO OPEN FILE AT `file_path` IN READ MODE  
    RETURN A LIST OF LINES WHERE EACH LINE IS SPLIT BY A COMMA AND STRIPPED OF WHITESPACE  
    IF FILE IS NOT FOUND (FileNotFoundException)  
    RETURN AN EMPTY LIST  
END FUNCTION `read_file`  
  
CALL FUNCTION `check_SUPpurchase_order` WITH `file_path`  
    CALL `read_file` WITH `file_path` AND ASSIGN RESULT TO `supplier_order_data`  
  
    PRINT "Purchase Order (SUPPLIER)"  
  
    FOR EACH `order` IN `supplier_order_data`  
        PRINT "Order ID:", `order[0]`, "Payment Status:", `order[4]`, "Delivery Status:", `order[3]`  
    ENDFOR  
  
    PRINT "***Example Order ID: order001"  
    PRINT "Enter Order ID to check status:"  
    SET `order_id` TO USER INPUT (STRIPPED OF WHITESPACE)  
  
    FOR EACH `order` IN `supplier_order_data`  
        IF `order[0]` EQUALS `order_id`  
            PRINT "Order ID:", `order[0]`, "Payment Status:", `order[4]`, "Delivery Status:", `order[3]`  
            BREAK OUT OF THE LOOP  
        ENDIF  
    ENDFOR  
  
    IF NO MATCHING ORDER ID WAS FOUND  
        PRINT "Order ID not found."  
    ENDIF  
  
END FUNCTION `check_SUPpurchase_order`  
  
CALL `check_SUPpurchase_order` WITH "SUP_purchase_orders.txt"
```

Flowchart



Function: Modify Purchase Order

```
START
  LOAD orders FROM file_path

  SET order_id TO input from user (trimmed)

  FOR each order IN orders
    IF order_id MATCHES order[0] THEN
      PRINT order details (Order ID, Item ID, Delivery Status, Payment Status, etc.)

      SET attribute_to_modify TO input from user (trimmed and lowercased)

      IF attribute_to_modify EQUALS "item id" THEN
        SET new_item_id TO input from user (trimmed)
        IF new_item_id EXISTS in inventory THEN
          SET order[2] TO new_item_id

          UPDATE unit price USING new_item_id AND UPDATE order[6]

          RECALCULATE price per item AND total price BASED ON new_item_id
          UPDATE order[7] AND order[10]
        ELSE
          PRINT "Invalid Item ID. Modification aborted."
          RETURN
        ENDIF
      ELSEIF attribute_to_modify EQUALS "quantity ordered" THEN
        SET new_quantity_ordered TO input from user (trimmed)
        IF new_quantity_ordered IS valid THEN
          SET order[5] TO new_quantity_ordered

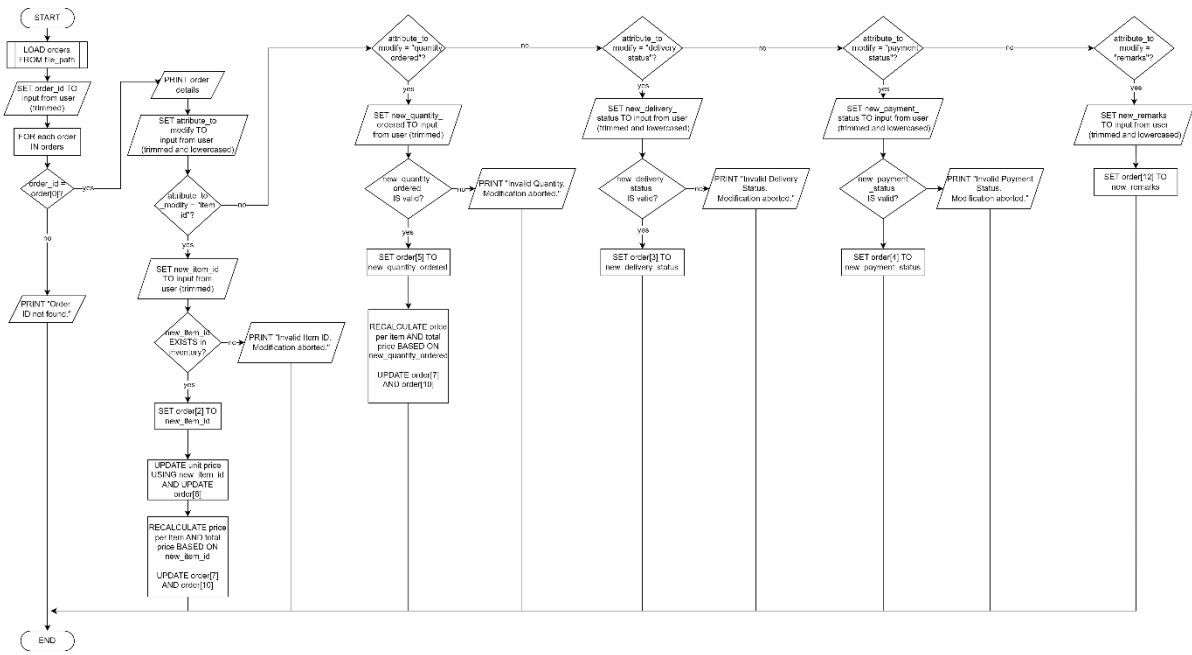
          RECALCULATE price per item AND total price BASED ON new_quantity_ordered
          UPDATE order[7] AND order[10]
        ELSE
          PRINT "Invalid Quantity. Modification aborted."
          RETURN
        ENDIF
      ELSEIF attribute_to_modify EQUALS "delivery status" THEN
        SET new_delivery_status TO input from user (trimmed and lowercased)
        IF new_delivery_status IS valid THEN
          SET order[3] TO new_delivery_status
        ELSE
          PRINT "Invalid Delivery Status. Modification aborted."
          RETURN
        ENDIF
      ELSEIF attribute_to_modify EQUALS "payment status" THEN
        SET new_payment_status TO input from user (trimmed and lowercased)
        IF new_payment_status IS valid THEN
          SET order[4] TO new_payment_status
        ELSE
          PRINT "Invalid Payment Status. Modification aborted."
          RETURN
        ENDIF
      ELSEIF attribute_to_modify EQUALS "remarks" THEN
        SET new_remarks TO input from user (trimmed)
        SET order[12] TO new_remarks
      ELSE
        PRINT "Invalid attribute. Modification aborted."
        RETURN
      ENDIF

      UPDATE order date AND time TO current date and time
      RECALCULATE expected_delivery_date AS 7 days from now
      UPDATE order[8], order[9], AND order[11]

      WRITE updated orders TO file_path
      PRINT "Order order_id has been modified."
      RETURN
    ENDIF
  ENDFOR

  PRINT "Order ID not found."
END
```

Flowchart



Function: Cancel Supplier Purchase Order

Pseudocode

Cancel Supplier Purchase Order

START

 LOAD orders FROM file_path

 SET order_id TO input from user (trimmed)

 FOR each order IN orders

 IF order_id MATCHES order[0] THEN

 PRINT order details (Order ID, Item ID, Delivery Status, Payment Status, etc.)

 SET confirm TO input from user (trimmed and lowercased)

 IF confirm EQUALS "yes" THEN

 REMOVE order FROM orders

 WRITE remaining orders TO file_path

 PRINT "Order order_id has been canceled."

 ELSE

 PRINT "Order cancellation aborted."

 ENDIF

 RETURN

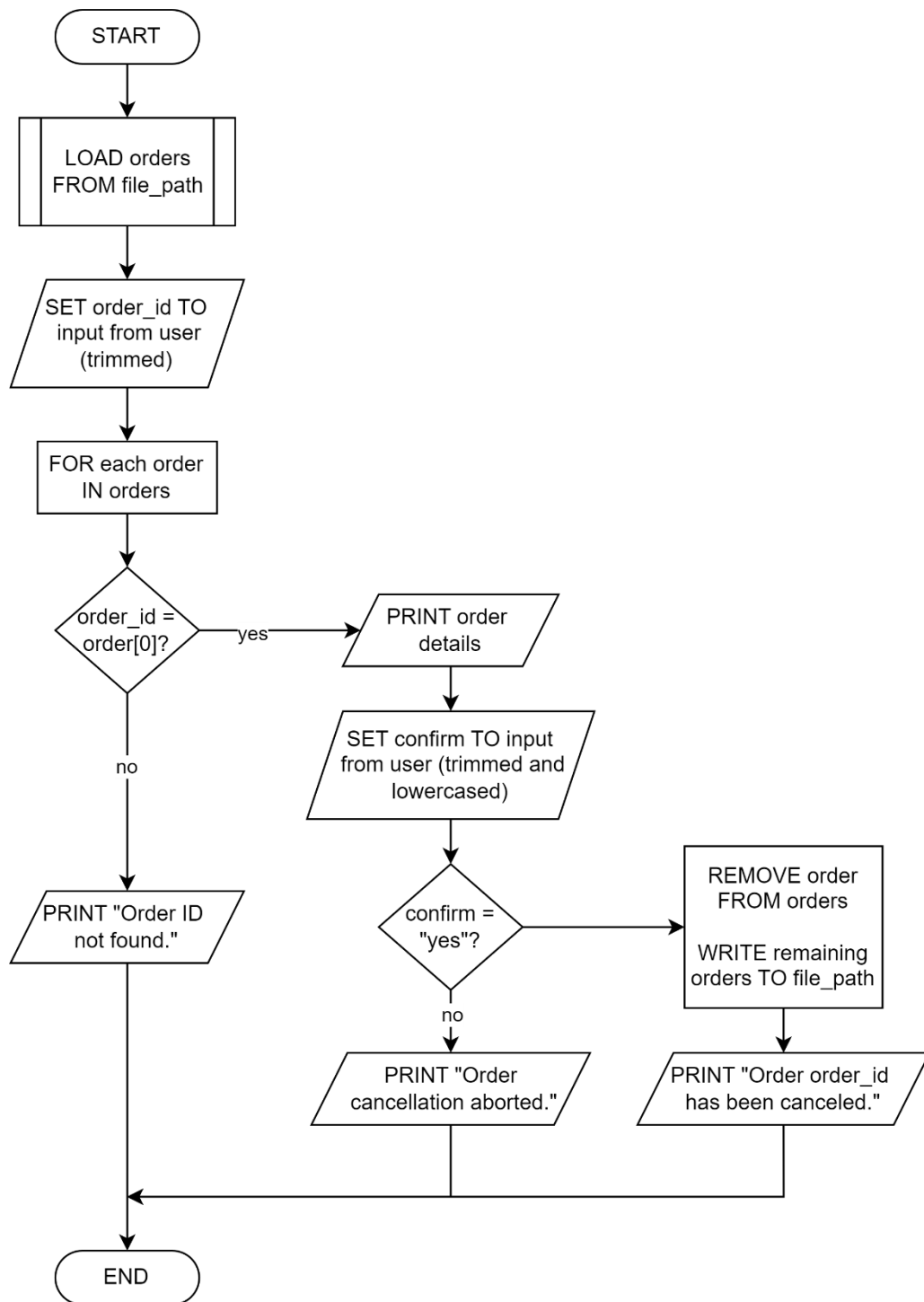
 ENDIF

 ENDFOR

 PRINT "Order ID not found."

END

Flowchart



Function: View Supplier Purchase Order

Pseudocode

View Supplier Purchase Order

START

LOAD orders FROM file_path

SET order_id TO input from user (trimmed)

FOR each order IN orders

IF order_id MATCHES order[0] THEN

PRINT order details (Order ID, Item ID, Delivery Status, Payment Status, etc.)

RETURN

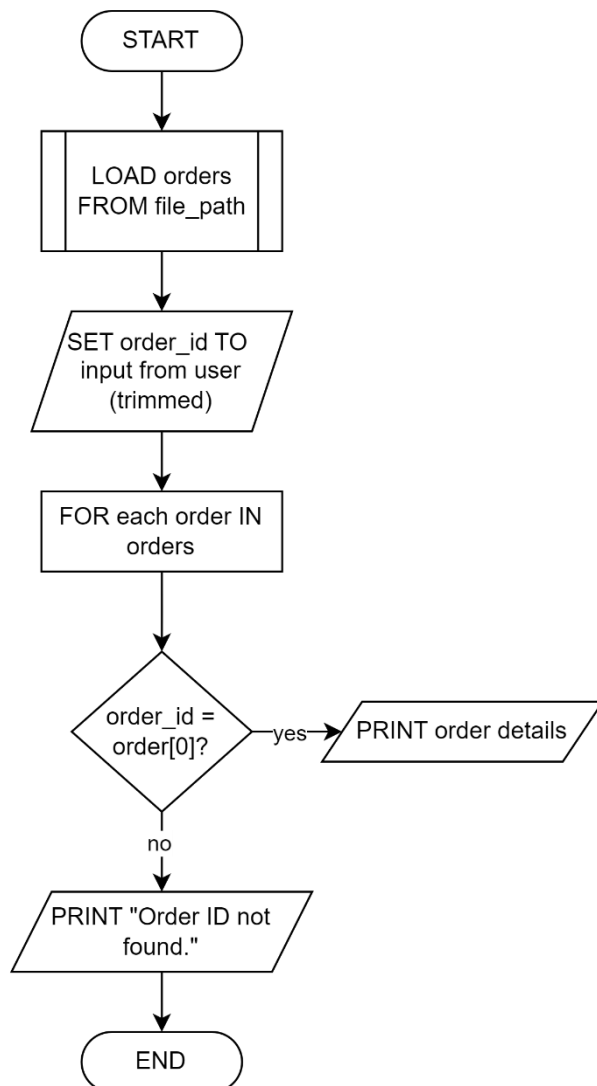
ENDIF

ENDFOR

PRINT "Order ID not found."

END

Flowchart



Function: Inventory Report

Pseudocode

Inventory Report

START

 LOAD inventory_list FROM "inventory_item.txt" USING load_inventory

 INITIALIZE inventory_report_list AS an empty list

 FOR each item IN inventory_list

 SET quantity_in_stock TO integer value of item[-1]

 SET unit_price TO float value of item[-2]

 CALCULATE total_value AS quantity_in_stock * unit_price

 APPEND total_value TO item FORMATTED to 2 decimal places

 ADD updated item TO inventory_report_list

 ENDFOR

 OPEN "inventory_report.txt" FOR writing AS file

 FOR each item IN inventory_report_list

 WRITE item TO file AS comma-separated values

 ENDFOR

 CLOSE file

 PRINT "Final Inventory Report:"

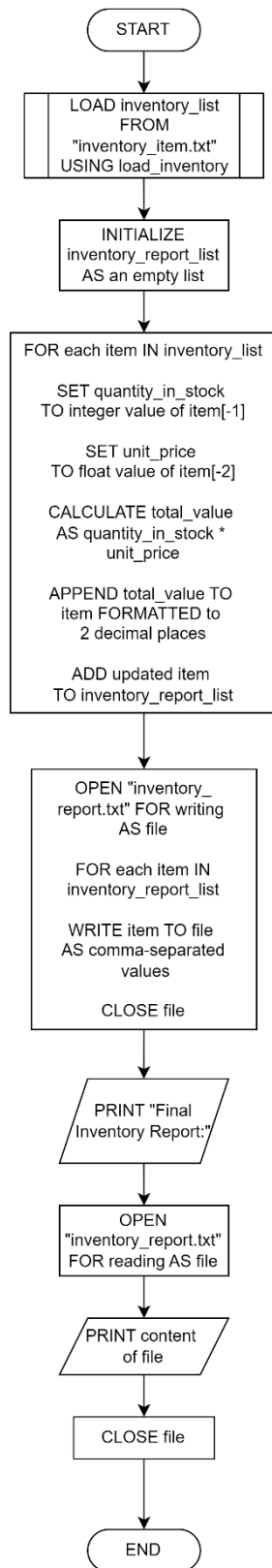
 OPEN "inventory_report.txt" FOR reading AS file

 PRINT content of file

 CLOSE file

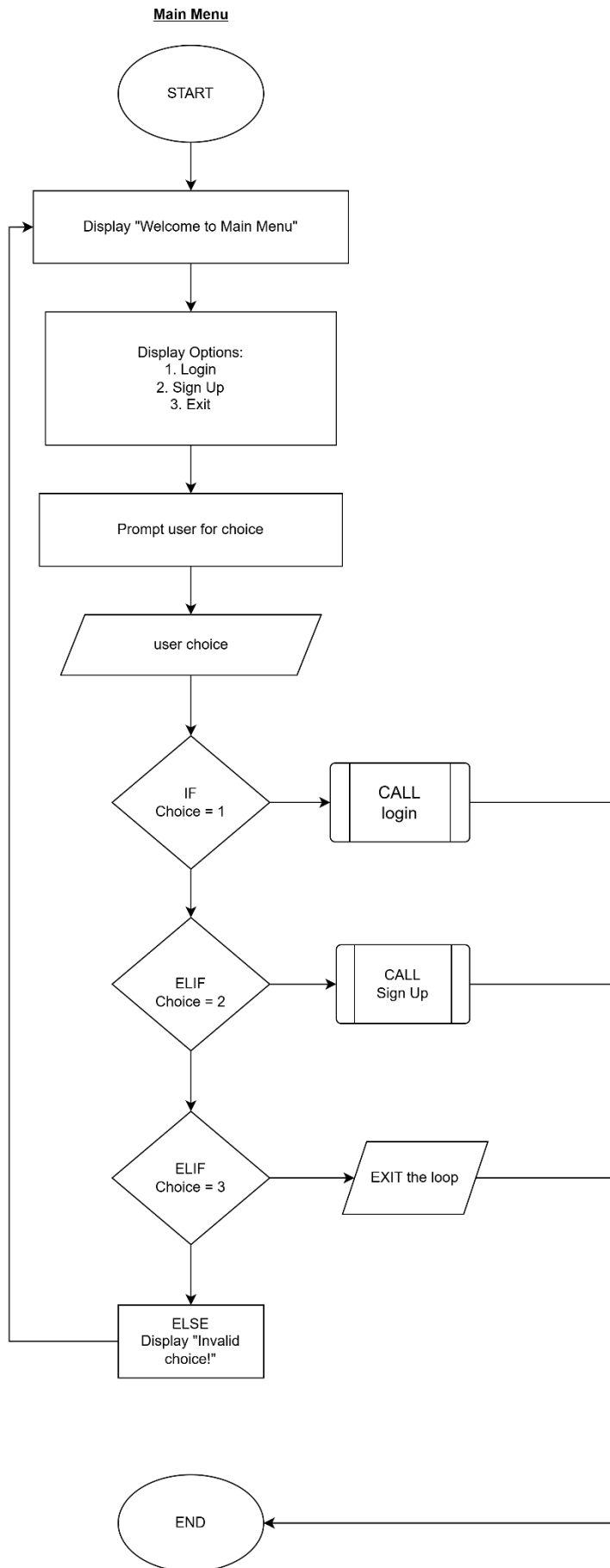
END

Flowchart



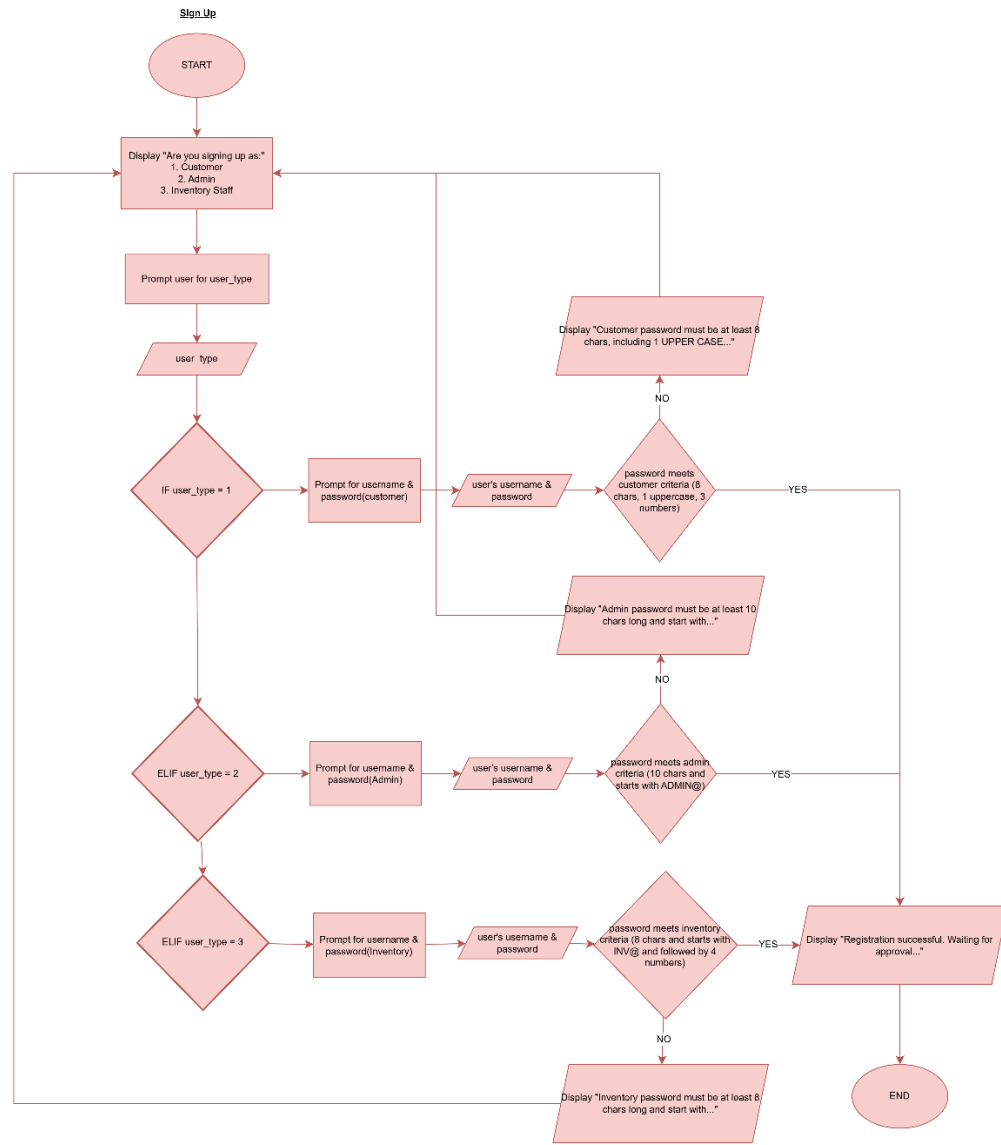
Function	Main Menu
Pseudocode	<pre> MAIN MENU ----- Display "Welcome to Main Menu" Loop: Display options: 1. Login 2. Sign Up 3. Exit Prompt user for choice If choice is "1": Call LOGIN FUNCTION Else If choice is "2": Call SIGN UP FUNCTION Else If choice is "3": Exit the loop Else: Display "Invalid choice" </pre>

Flowchart



Function	Sign up
Pseudocode	<pre> FUNCTION sign_up DISPLAY "Are you signing up as:" DISPLAY "1. Customer" DISPLAY "2. Admin" DISPLAY "3. Inventory Staff" PROMPT user for user_type IF user_type is not "1", "2", or "3" THEN DISPLAY "Invalid selection!" RETURN END IF login_data = READ_FILE(login_data_file) PROMPT user for username username = user input PROMPT user for password password = user input IF user_type is "1" THEN // Customer IF password does not meet criteria THEN DISPLAY "Customer password must be at least 8 characters long, including 1 UPPER CASE and 3 NUMBERS." RETURN END IF customer_data = [username, password, "Pending", user_type] APPEND customer_data to login_data ELSE IF user_type is "2" THEN // Admin IF password does not meet criteria THEN DISPLAY "Admin password must be at least 10 characters long and start with 'ADMIN@'." RETURN END IF APPEND [username, password, "Pending", user_type] to login_data ELSE IF user_type is "3" THEN // Inventory Staff IF password does not meet criteria THEN DISPLAY "Inventory staff password must be at least 8 characters long, start with 'INV@' and followed by 4 numbers." RETURN END IF APPEND [username, password, "Pending", user_type] to login_data END IF WRITE_FILE(login_data_file, login_data) DISPLAY "Registration successful. Waiting for approval." END FUNCTION </pre>

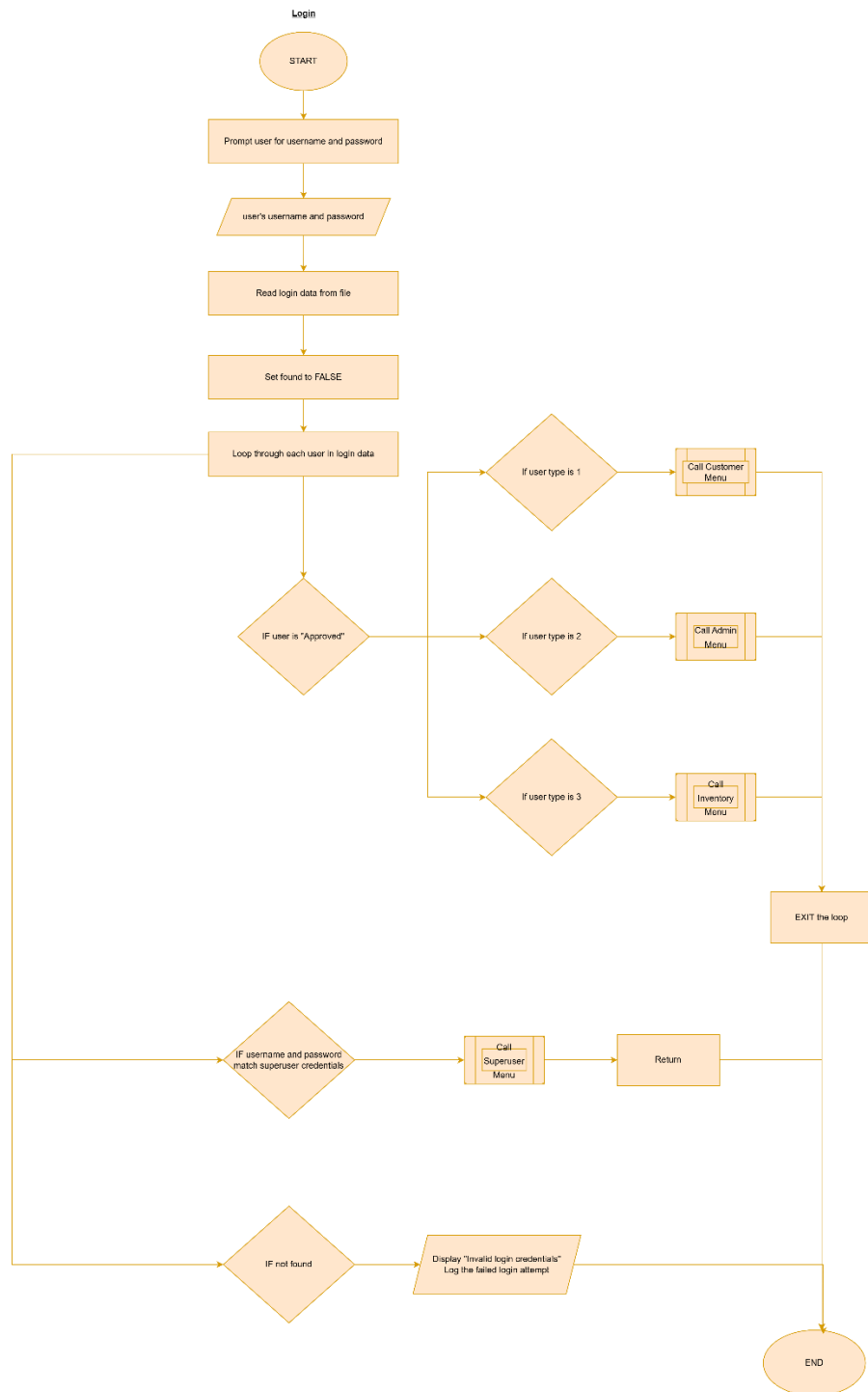
Flowchart



Function	Login
Pseudocode	<pre> LOGIN FUNCTION ----- Prompt user for username and password Read login data from file Set found to False Loop through each user in login data: If username and password match current user's username and password: Set found to True If user is "Approved": If user type is "1": Call CUSTOMER MENU Else If user type is "2": Call ADMIN MENU Else If user type is "3": Call INVENTORY MENU Exit the loop If username and password match superuser credentials: Call SUPERUSER MENU Return If not found: Display "Invalid login credentials" Log the failed login attempt CUSTOMER MENU ----- Display "Welcome to Customer Menu" Loop: Display options: 1. Purchase Order 2. Repair/Service Order 3. Modify Purchase Order/Repair/Service Order 4. Payment for orders placed 5. Inquiry of order status 6. Cancel Order 7. Reports 8. Logout Prompt user for choice </pre>

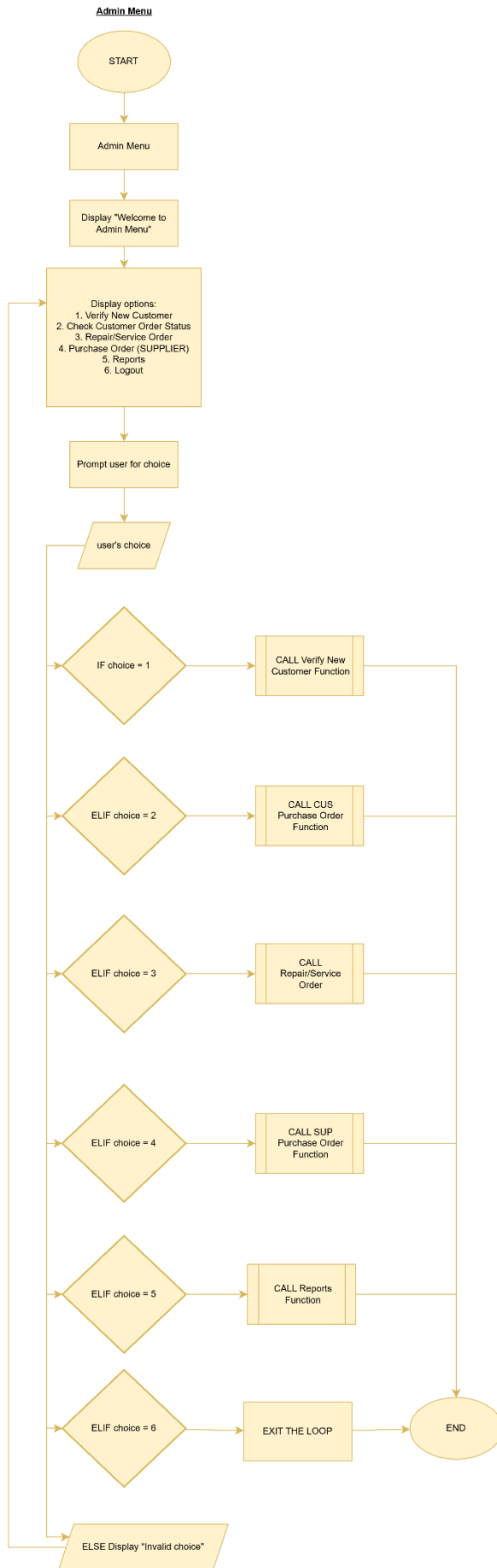
	If choice is "1": Call PURCHASE ORDER FUNCTION Else If choice is "2": Call SERVICE/REPAIR ORDER FUNCTION Else If choice is "3": Call MODIFY ORDER FUNCTION Else If choice is "4": Call MAKE PAYMENT FUNCTION Else If choice is "5": Call INQUIRE ORDER STATUS FUNCTION Else If choice is "6": Call CANCEL ORDER FUNCTION Else If choice is "7": Call VIEW REPORTS FUNCTION Else If choice is "8": Break loop (Logout) Else: Display "Invalid choice"
--	---

Flowchart



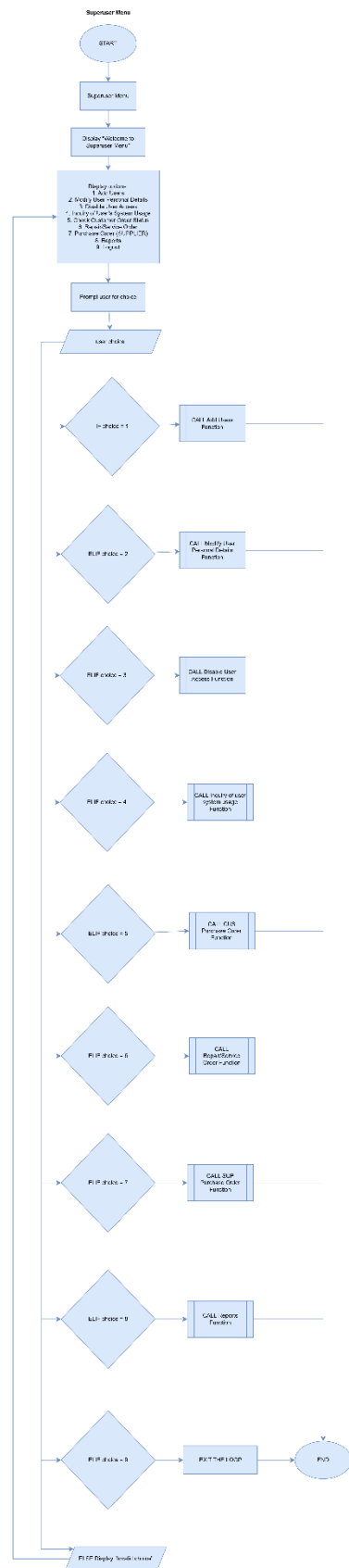
Function	Admin Menu
Pseudocode	<pre> ADMIN MENU ----- Display "Welcome to Admin Menu" Loop: Display options: 1. Verify New Customer 2. Check Customer Order Status 3. Repair/Service Order 4. Purchase Order (SUPPLIER) 5. Reports 6. Logout Prompt user for choice If choice is "1": Call VERIFY NEW CUSTOMER FUNCTION Else If choice is "2": Call CUS PURCHASE ORDER FUNCTION Else If choice is "3": Call REPAIR/SERVICE ORDER FUNCTION Else If choice is "4": Call SUP PURCHASE ORDER FUNCTION Else If choice is "5": Call REPORTS FUNCTION Else If choice is "6": Break loop (Logout) Else: Display "Invalid choice" </pre>

Flowchart



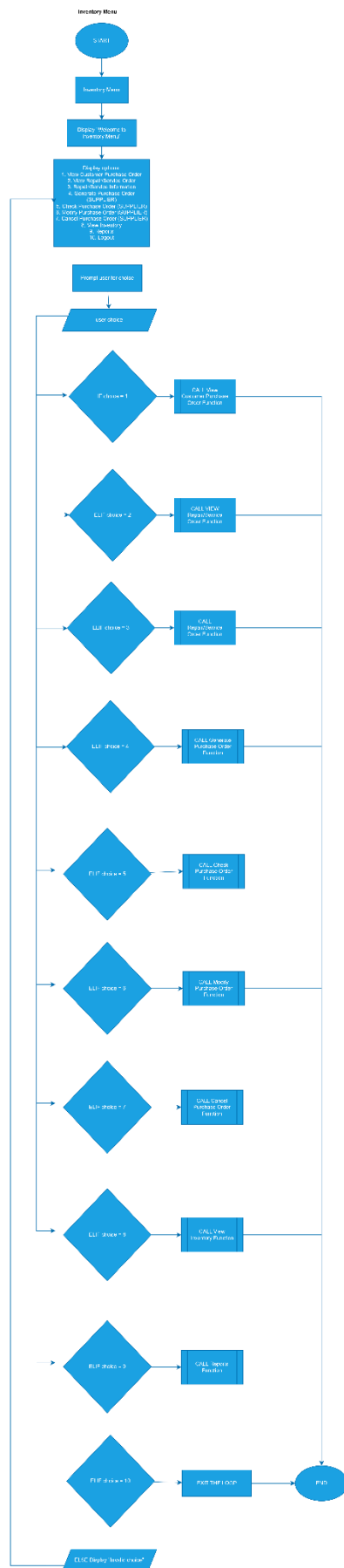
Function	Superuser Menu
Pseudocode	<pre> SUPERUSER MENU ----- Display "Welcome to Superuser Menu" Loop: Display options: 1. Add Users 2. Modify User Personal Details 3. Disable User Access 4. Inquiry of User's System Usage 5. Check Customer Order Status 6. Repair/Service Order 7. Purchase Order (SUPPLIER) 8. Reports 9. Logout Prompt user for choice If choice is "1": Call ADD USERS FUNCTION Else If choice is "2": Call MODIFY USER PERSONAL DETAILS FUNCTION Else If choice is "3": Call DISABLE USER ACCESS FUNCTION Else If choice is "4": Call INQUIRY OF USER SYSTEM USAGE FUNCTION Else If choice is "5": Call CUS PURCHASE ORDER FUNCTION Else If choice is "6": Call REPAIR/SERVICE ORDER FUNCTION Else If choice is "7": Call SUP PURCHASE ORDER FUNCTION Else If choice is "8": Call REPORTS FUNCTION Else If choice is "9": Break loop (Logout) Else: Display "Invalid choice" </pre>

Flowchart



Function	Inventory Menu
Pseudocode	<pre> INVENTORY MENU ----- Display "Welcome to Inventory Menu" Loop: Display options: 1. View Customer Purchase Order 2. View Repair/Service Order 3. Repair/Service Information 4. Generate Purchase Order (SUPPLIER) 5. Check Purchase Order (SUPPLIER) 6. Modify Purchase Order (SUPPLIER) 7. Cancel Purchase Order (SUPPLIER) 8. View Inventory 9. Reports 10. Logout Prompt user for choice If choice is "1": Call VIEW CUSTOMER PURCHASE ORDER FUNCTION Else If choice is "2": Call VIEW REPAIR/SERVICE ORDER FUNCTION Else If choice is "3": Call REPAIR/SERVICE INFORMATION FUNCTION Else If choice is "4": Call GENERATE PURCHASE ORDER FUNCTION Else If choice is "5": Call CHECK PURCHASE ORDER FUNCTION Else If choice is "6": Call MODIFY PURCHASE ORDER FUNCTION Else If choice is "7": Call CANCEL PURCHASE ORDER FUNCTION Else If choice is "8": Call VIEW INVENTORY FUNCTION Else If choice is "9": Call REPORTS FUNCTION Else If choice is "10": Break loop (Logout) Else: Display "Invalid choice" </pre>

Flowchart

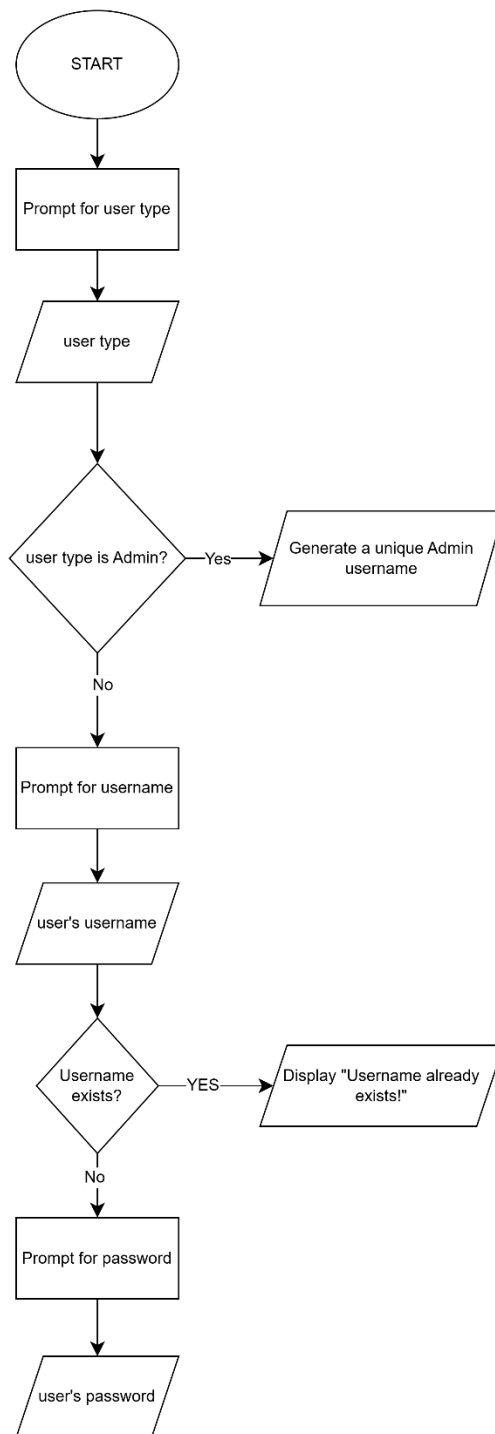


Function	Add users function
Pseudocode	<pre> FUNCTION add_users PROMPT user for user type IF user type is Admin THEN GENERATE a unique Admin username ELSE PROMPT user for username IF username already exists THEN DISPLAY "Username already exists!" RETURN END IF END IF PROMPT user for password APPEND new user data to login_data with "Approved" status WRITE updated login_data to login_data_file DISPLAY "User added successfully" END FUNCTION FUNCTION modify_user_personal_details PROMPT for username of user to modify READ login_data from login_data_file IF username is found in login_data THEN DISPLAY current user details LOOP DISPLAY options to modify: 1. Username 2. Password 3. Approval Status 4. User Type 5. Exit PROMPT user for choice IF choice is "1" THEN PROMPT for new username IF new username already exists THEN DISPLAY "Username already exists!" ELSE UPDATE username END IF ELSE IF choice is "2" THEN PROMPT for new password VALIDATE new password based on user type IF new password does not meet criteria THEN DISPLAY "Password does not meet criteria." ELSE UPDATE password END IF ELSE IF choice is "3" THEN PROMPT for new approval status UPDATE approval status ELSE IF choice is "4" THEN PROMPT for new user type IF new user type is valid THEN UPDATE user type ELSE </pre>

	<pre> DISPLAY "Invalid user type." END IF ELSE IF choice is "5" THEN BREAK loop (Exit) ELSE DISPLAY "Invalid choice or not applicable" END LOOP WRITE updated login_data to login_data_file DISPLAY "User details updated successfully" ELSE DISPLAY "User not found" END IF END FUNCTION</pre>
--	---

Flowchart

Add Users Function



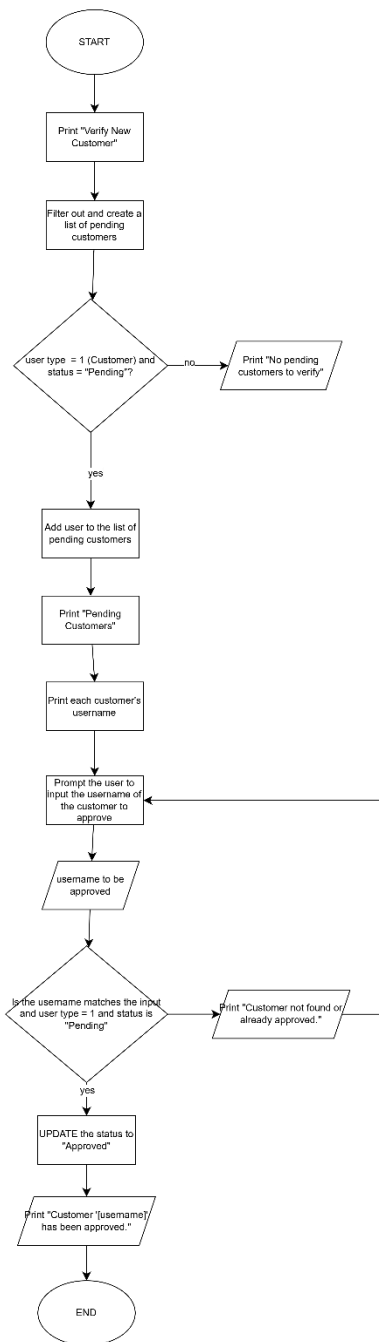
Function	Disable User Access Function
Pseudocode	DISABLE USER ACCESS FUNCTION ----- Prompt for username of user to disable If username found in login data: Remove user from login data file Display "User access disabled and user removed" Else: Display "User not found"
Flowchart	<u>Disable User Access Function</u> <pre> graph TD Start([START]) --> Prompt[Prompt for username of user to disable] Prompt --> Input[/User's username to disable/] Input --> Decision{Username found in login data?} Decision -- no --> DisplayNotFound[/Display "User not found"/] DisplayNotFound --> Prompt Decision -- yes --> RemoveUser[Remove user from login data file] RemoveUser --> DisplaySuccess[/Display "User access disabled and user removed"/] DisplaySuccess --> End([END]) </pre>

Function	Inquiry of user system usage function
Pseudocode	INQUIRY OF USER SYSTEM USAGE FUNCTION Prompt for username to inquire about If activity found for username in activity log: Display user activity Else: Display "No activity found for this user"
Flowchart	<u>Inquiry of user system usage function</u> <pre> graph TD START([START]) --> Prompt[Prompt for username to inquire about] Prompt --> Input[/username to inquire about/] Input --> Decision{Activity found for username in activity log?} Decision -- No --> OutputNo[/Display "No activity found for this user"/] OutputNo --> Prompt Decision -- Yes --> OutputYes[/Display user activity/] OutputYes --> END([END]) </pre>

Function	Verify new customer function
Pseudocode	<pre> VERIFY NEW CUSTOMER FUNCTION ----- Print "Verify New Customer" Read the login data from the file Filter out and create a list of pending customers: Loop through each user in the login data: If user type is "1" (Customer) and status is "Pending": Add user to the list of pending customers If the list of pending customers is empty: Print "No pending customers to verify." Return from the function Print "Pending Customers:" Loop through the list of pending customers: Print each customer's username Prompt the user to input the username of the customer to approve Loop through each user in the login data: If the username matches the input and user type is "1" and status is "Pending": Update the status to "Approved" Write the updated login data back to the file Print "Customer '[username]' has been approved." Return from the function If no match is found: Print "Customer not found or already approved." Write updated login data to file Display "Customer verification process completed" </pre>

Flowchart

Verify New Customer Function

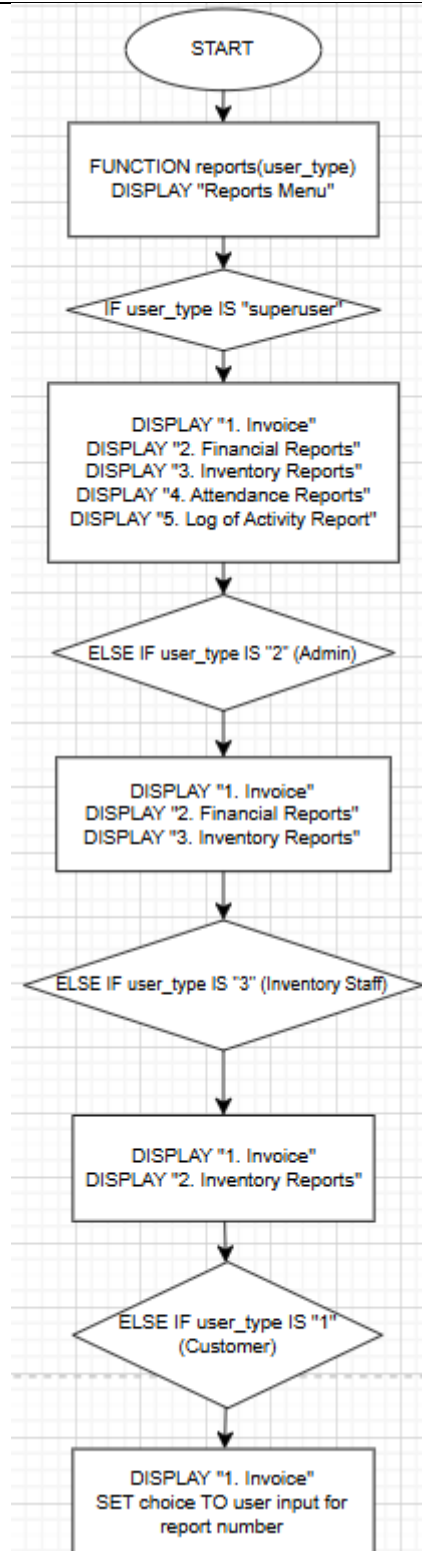


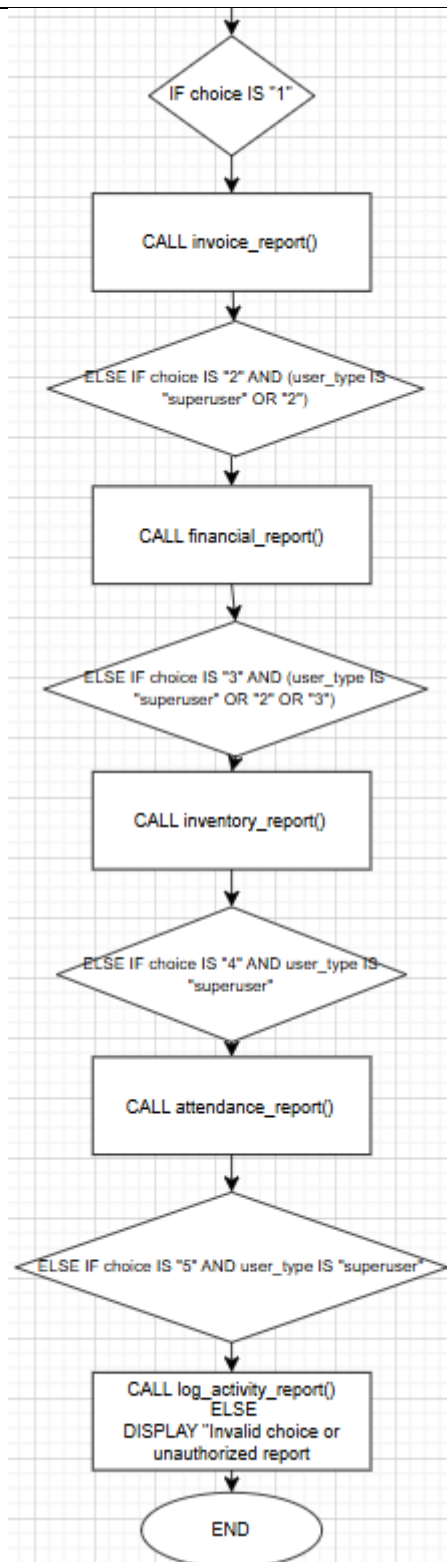
Function	Customer Purchase Order Function
Pseudocode	CUS PURCHASE ORDER FUNCTION ----- Display all customer orders and their statuses Prompt for Order ID to check status If Order ID found: Display status of the order Else: Display "Order ID not found"
Flowchart	<u>CUS Purchase Order Function</u> <pre>graph TD; START([START]) --> DisplayOrders[Display all customer orders and their statuses]; DisplayOrders --> PromptID[Prompt for Order ID to check status]; PromptID --> OrderID[/Order ID/]; OrderID --> IsFound{Is Order ID found?}; IsFound -- no --> DisplayNotFound[/Display "order ID not found"/]; DisplayNotFound --> START; IsFound --> DisplayStatus[/Display all customer orders and their statuses/]; DisplayStatus --> END([END]);</pre> The flowchart illustrates the process of checking an order status. It begins with a 'START' terminal, followed by a process box 'Display all customer orders and their statuses'. This leads to another process box 'Prompt for Order ID to check status', which then leads to an input/output box 'Order ID'. A decision diamond 'Is Order ID found?' follows. If the answer is 'no', the flow goes to an output box 'Display "order ID not found"' and loops back to the 'START' terminal. If the answer is 'yes' (implied by the downward arrow), the flow goes to an output box 'Display all customer orders and their statuses' and then to an 'END' terminal.

Function	Repair/Service Order Function
Pseudocode	REPAIR/SERVICE ORDER FUNCTION ----- Display all repair/service orders and their statuses Prompt for Repair ID to check status If Repair ID found: Display status of the repair/service order Else: Display "Repair ID not found"
Flowchart	<u>Repair/Service Order Function</u> <pre>graph TD; START([START]) --> DisplayOrders[Display all supplier orders and their statuses]; DisplayOrders --> PromptID[Prompt for Repair ID to check status]; PromptID --> InputID[/Repair ID/]; InputID --> Decision{Is Repair ID be found?}; Decision -- no --> DisplayNotFound[/Display "Repair ID not found"/]; DisplayNotFound --> START; Decision -- yes --> DisplayStatus[/Display status of the repair/service order/]; DisplayStatus --> END([END]);</pre> The flowchart illustrates the logic of the Repair/Service Order Function. It begins with a 'START' terminal, followed by a process to 'Display all supplier orders and their statuses'. Then, it prompts for a 'Repair ID to check status' and receives the 'Repair ID' as input. A decision diamond asks 'Is Repair ID be found?'. If the answer is 'no', the flow goes to 'Display "Repair ID not found"' and loops back to the 'START' terminal. If the answer is 'yes', the flow proceeds to 'Display status of the repair/service order' and then to the 'END' terminal.

Function	Reports Function
Pseudocode	<pre> FUNCTION reports(user_type): DISPLAY "Reports Menu" IF user_type IS "superuser": DISPLAY "1. Invoice" DISPLAY "2. Financial Reports" DISPLAY "3. Inventory Reports" DISPLAY "4. Attendance Reports" DISPLAY "5. Log of Activity Report" ELSE IF user_type IS "2": # Admin DISPLAY "1. Invoice" DISPLAY "2. Financial Reports" DISPLAY "3. Inventory Reports" ELSE IF user_type IS "3": # Inventory Staff DISPLAY "1. Invoice" DISPLAY "2. Inventory Reports" ELSE IF user_type IS "1": # Customer DISPLAY "1. Invoice" SET choice TO user input for report number IF choice IS "1": CALL invoice_report() ELSE IF choice IS "2" AND user_type IS "superuser" OR "2": CALL financial_report() ELSE IF choice IS "3" AND user_type IS "superuser" OR "2" OR "3": CALL inventory_report() ELSE IF choice IS "4" AND user_type IS "superuser": CALL attendance_report() ELSE IF choice IS "5" AND user_type IS "superuser": CALL log_activity_report() ELSE: DISPLAY "Invalid choice or unauthorized report." </pre>

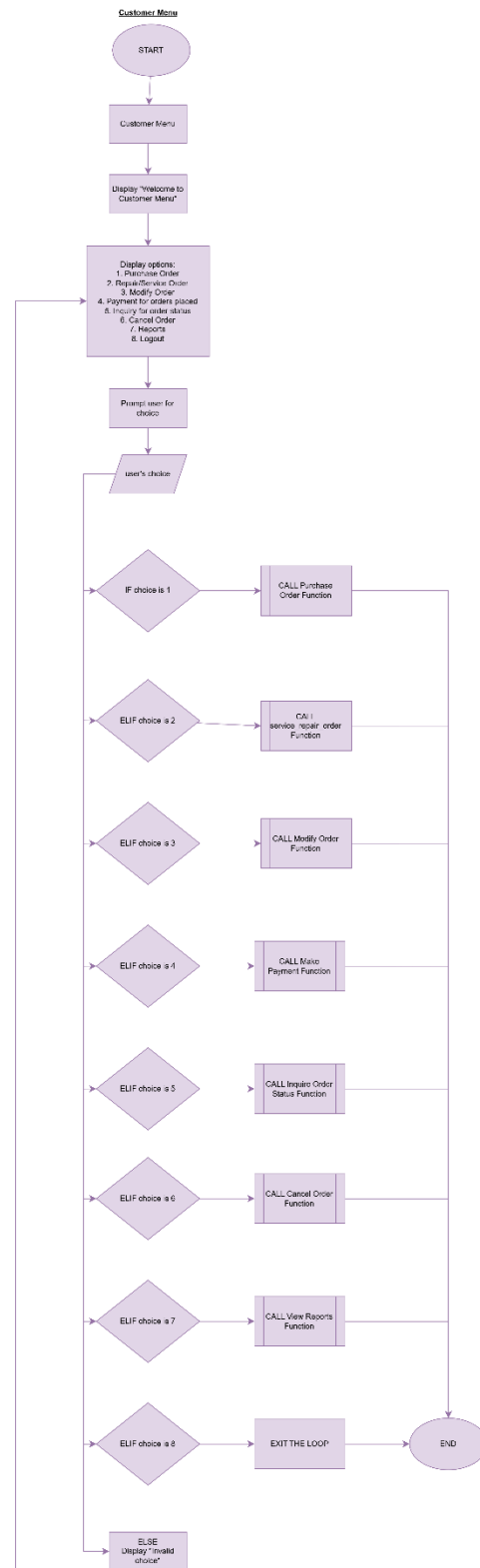
Flowchart





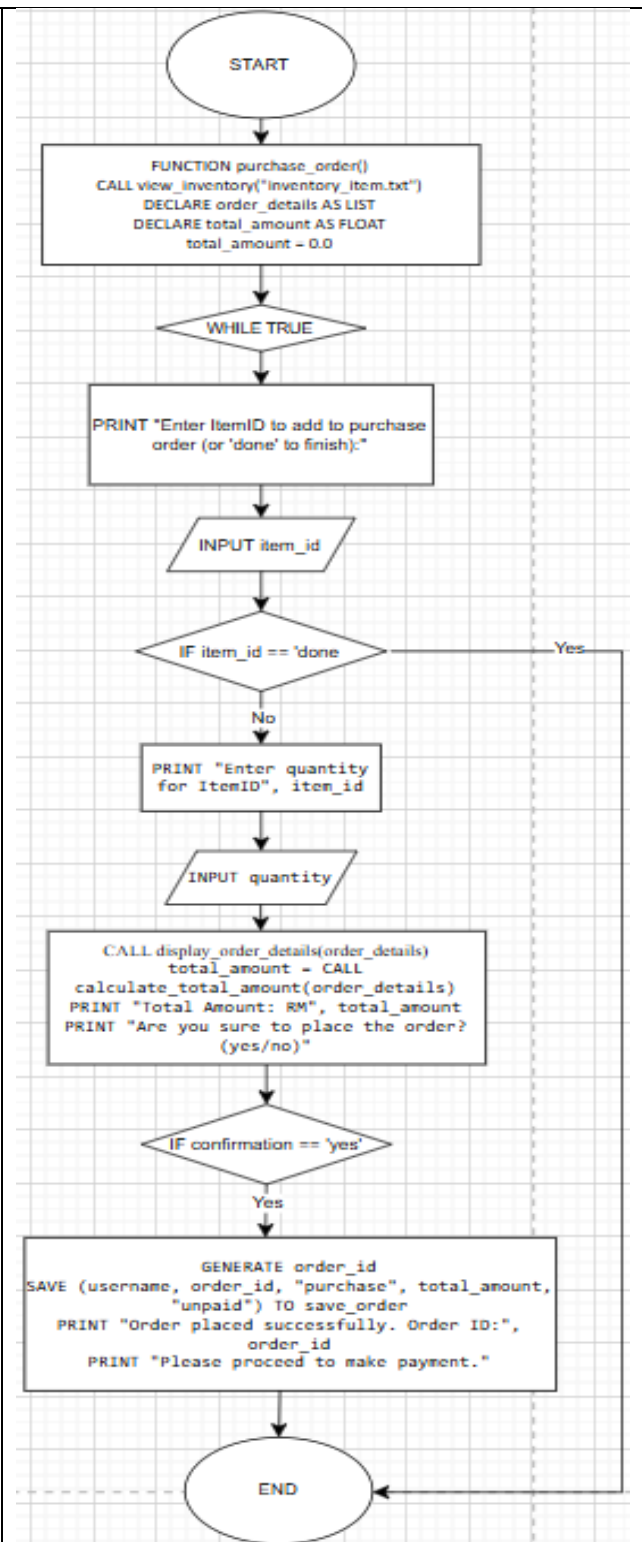
Function	Customer Menu
Pseudocode	CUSTOMER MENU ----- Display "Welcome to Customer Menu" Loop: Display options: 1. Purchase Order 2. Repair/Service Order 3. Modify Purchase Order/Repair/Service Order 4. Payment for orders placed 5. Inquiry of order status 6. Cancel Order 7. Reports 8. Logout Prompt user for choice If choice is "1": Call PURCHASE ORDER FUNCTION Else If choice is "2": Call service_repair_order FUNCTION Else If choice is "3": Call MODIFY ORDER FUNCTION Else If choice is "4": Call MAKE PAYMENT FUNCTION Else If choice is "5": Call INQUIRE ORDER STATUS FUNCTION Else If choice is "6": Call CANCEL ORDER FUNCTION Else If choice is "7": Call VIEW REPORTS FUNCTION Else If choice is "8": Break loop (Logout) Else: Display "Invalid choice"

Flowchart



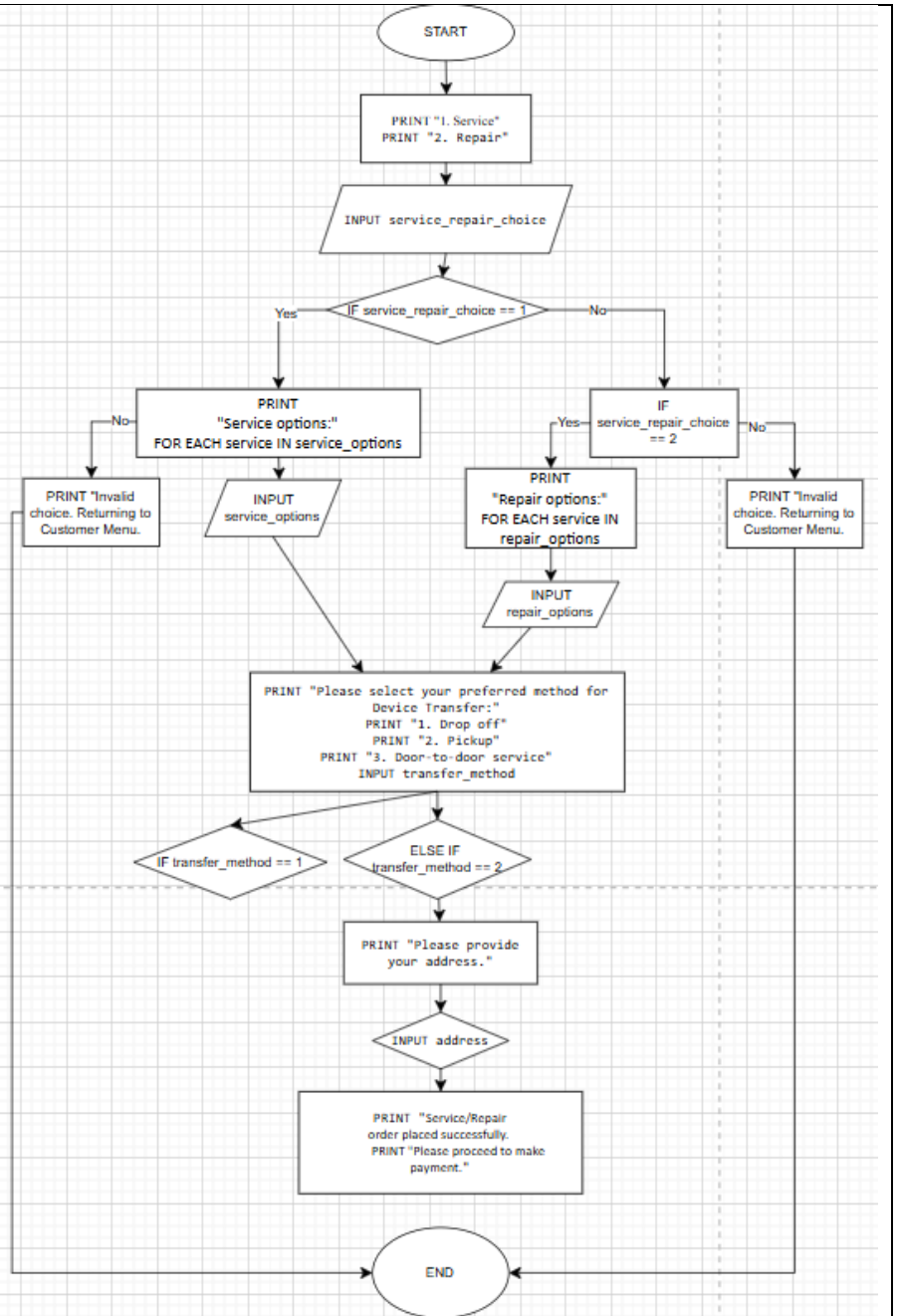
Function	Purchase order
Pseudocode	<pre> FUNCTION purchase_order() CALL view_inventory("inventory_item.txt") DECLARE order_details AS LIST DECLARE total_amount AS FLOAT total_amount = 0.0 WHILE TRUE PRINT "Enter ItemID to add to purchase order (or 'done' to finish):" INPUT item_id IF item_id == 'done' BREAK END IF PRINT "Enter quantity for ItemID", item_id INPUT quantity ADD (item_id, quantity) TO order_details CALL display_order_details(order_details) total_amount = CALL calculate_total_amount(order_details) PRINT "Total Amount: RM", total_amount PRINT "Are you sure to place the order? (yes/no)" INPUT confirmation IF confirmation == 'yes' GENERATE order_id SAVE (username, order_id, "purchase", total_amount, "unpaid") TO save_order PRINT "Order placed successfully. Order ID:", order_id PRINT "Please proceed to make payment." RETURN END IF END WHILE END FUNCTION </pre>

Flowchart



Function	Service repair order
Pseudocode	<pre> FUNCTION service_repair_order() PRINT "1. Service" PRINT "2. Repair" INPUT service_repair_choice IF service_repair_choice == 1 PRINT: "Service options:" FOR EACH service IN service_options ELSE IF service_repair_choice == 2 PRINT: "Repair options:" FOR EACH srepair IN repair_options ELSE PRINT "Invalid choice. Returning to Customer Menu." RETURN END IF PRINT "Please select your preferred method for Device Transfer:" PRINT "1. Drop off" PRINT "2. Pickup" INPUT transfer_method INPUT confirmation IF confirmation == 'yes' GENERATE order_id SAVE (username, order_id, "service/repair", total_amount, "unpaid") TO save_order PRINT "Service/Repair order placed successfully. Order ID:", order_id PRINT "Please proceed to make payment." END IF END FUNCTION </pre>

Flowchart

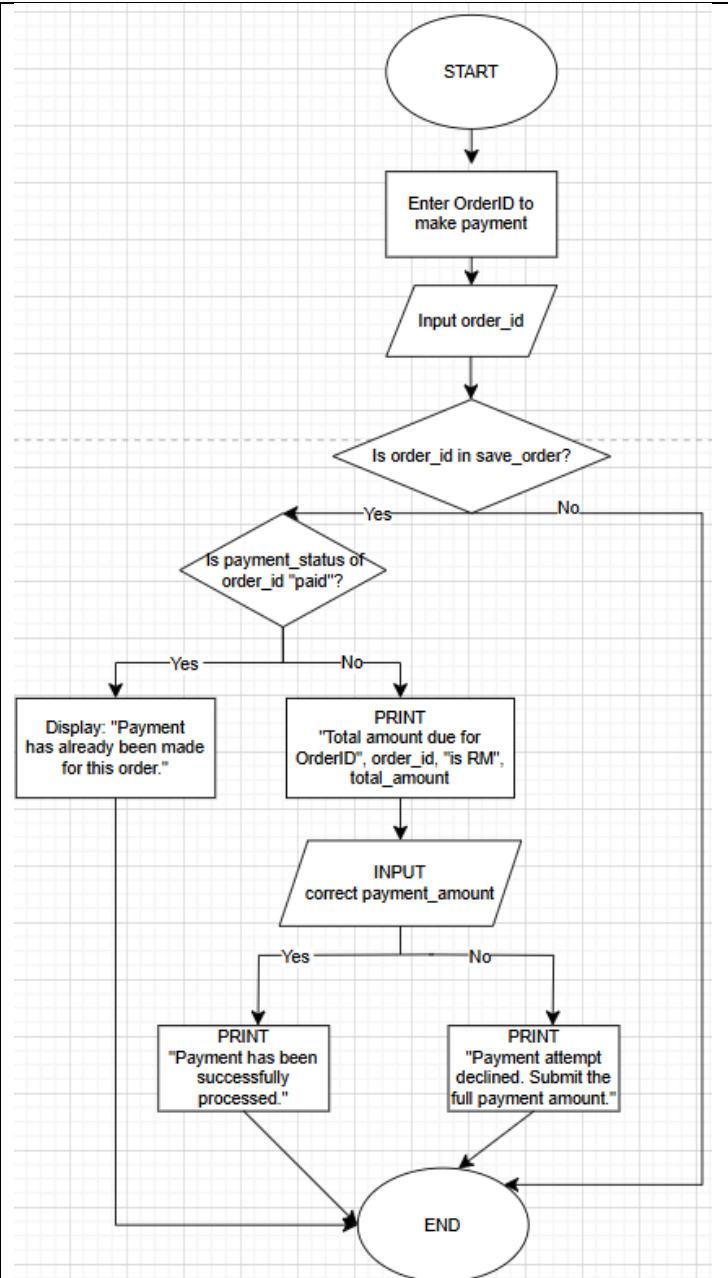


Function	Modify order
Pseudocode	<pre> FUNCTION modify_order() PRINT "Enter OrderID to modify:" INPUT order_id CHECK order_id IN save_order IF NOT FOUND PRINT "OrderID does not exist." RETURN END IF CHECK payment_status OF order_id IF payment_status == "paid" PRINT "Cannot modify the order as payment has already been made." RETURN END IF PRINT "Choose an option to modify:" PRINT "1. Purchase Order" PRINT "2. Service/Repair Order" INPUT modify_choice IF modify_choice == 1 CALL modify_purchase_order(order_id) ELSE IF modify_choice == 2 CALL modify_service_repair_order(order_id) ELSE PRINT "Invalid choice. Returning to Customer Menu." RETURN END IF END FUNCTION </pre>
Flowchart	<pre> graph TD START([START]) --> INPUT_order_id[/INPUT order_id/] INPUT_order_id -- Yes --> CHECK_order_id{CHECK order_id IN save_order} CHECK_order_id -- No --> END([END]) CHECK_order_id -- Yes --> CHECK_payment_status{CHECK payment_status of order_id} CHECK_payment_status -- Paid --> END CHECK_payment_status -- Unpaid --> PRINT_choose[PRINT "Choose order type to modify:" PRINT "1. Purchase Order" PRINT "2. Service/Repair Order"] PRINT_choose --> INPUT_modify_choice[/INPUT modify_choice/] INPUT_modify_choice --> IF_modify_choice_1{IF modify_choice == 1} INPUT_modify_choice --> IF_modify_choice_2{IF modify_choice == 2} IF_modify_choice_1 --> MODIFY_purchase[Modify the purchase order with the new details] IF_modify_choice_2 --> MODIFY_service[Modify the service/repair order with the new details] MODIFY_purchase --> DISPLAY_DETAILS[CALL display_order_details(order_details) total_amount = CALL calculate_total_amount(order_details) PRINT "Total Amount: RM", total_amount PRINT "Are you sure to save the modifications?" (yes/no) INPUT confirmation] MODIFY_service --> DISPLAY_DETAILS DISPLAY_DETAILS --> IF_confirmation{IF confirmation == 'yes'} IF_confirmation -- Yes --> UPDATE_order[UPDATE order_details IN save_order PRINT "Order modified successfully."] IF_confirmation -- No --> END UPDATE_order --> END </pre>

Function	Cancel order
Pseudocode	<pre> FUNCTION cancel_order() PRINT "Enter OrderID to cancel:" INPUT order_id CHECK order_id IN save_order IF FOUND DELETE order_id FROM save_order PRINT "OrderID", order_id, "has been canceled." ELSE PRINT "OrderID does not exist." END IF END FUNCTION </pre>
Flowchart	<pre> graph TD Start([START]) --> Process1[FUNCTION cancel_order() PRINT "Enter OrderID to cancel." INPUT order_id] Process1 --> Decision1{CHECK order_id IN save_order} Decision1 -- Yes --> Process2[DELETE order_id FROM save_order PRINT "OrderID", order_id, "has been canceled."] Decision1 -- ELSE --> Process3[PRINT "OrderID does not exist."] Process2 --> End([END]) Process3 --> End </pre>

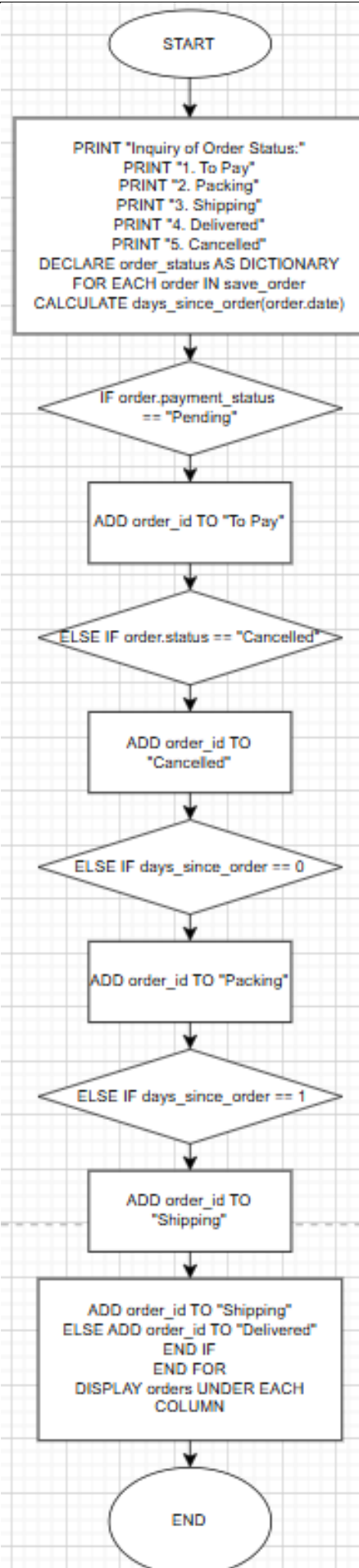
Function	Make payment
Pseudocode	<pre> FUNCTION make_payment() PRINT "Enter OrderID to make payment:" INPUT order_id CHECK order_id IN save_order IF NOT FOUND PRINT "OrderID does not exist." RETURN END IF CHECK payment_status OF order_id IF payment_status == "paid" PRINT "Payment has already been made for this order." RETURN END IF PRINT "Total amount due for OrderID", order_id, "is RM", total_amount INPUT payment_amount IF payment_amount > total_amount PRINT "Payment amount exceeds the total due." RETURN ELSE IF payment_amount == total_amount MARK order_id AS "paid" PRINT "Payment has been successfully processed." ELSE PRINT "Payment attempt declined. Submit the full payment amount." RETURN END IF END FUNCTION </pre>

Flowchart



Function	Inquire order status
Pseudocode	<pre> FUNCTION inquire_order_status() PRINT "Inquiry of Order Status:" PRINT "1. To Pay" PRINT "2. Packing" PRINT "3. Shipping" PRINT "4. Delivered" PRINT "5. Cancelled" DECLARE order_status AS DICTIONARY FOR EACH order IN save_order CALCULATE days_since_order(order.date) IF order.payment_status == "Pending" ADD order_id TO "To Pay" ELSE IF order.status == "Cancelled" ADD order_id TO "Cancelled" ELSE IF days_since_order == 0 ADD order_id TO "Packing" ELSE IF days_since_order == 1 ADD order_id TO "Shipping" ELSE ADD order_id TO "Delivered" END IF END FOR DISPLAY orders UNDER EACH COLUMN END FUNCTION </pre>

Flowchart



Program source code

TASK
The user must sign up for the system. If the new user is a customer , he/she will be approved by regular admin staff or a Super user . If the new user is an admin , he/she will be approved by the Super User .

SOURCE CODE	EXPLANATION
<pre>import datetime import json import os # File paths login_data_file = "login_data.txt" customer_data_file = "customer_data.txt" inventory_data_file = "inventory_data.txt" repair_data_file = "repair_data.txt" supplier_order_data_file = "supplier_order_data.txt" user_activity_log_file = "user_activity_log.txt" load_sup_purchase_order = "supplier_orders.txt" inventory_file = 'inventory.json' customer_data_file = 'customer_data.json' order_file = 'orders.json' # Sample users for demonstration superuser_credentials = ["superuser@123", "superusernotsuperman"] def read_file(file_path): try: with open(file_path, "r") as file: return [line.strip().split(",") for line in file.readlines()] except FileNotFoundError: return [] def write_file(file_path, data): with open(file_path, "w") as file: for line in data: file.write(",".join(line) + "\n")</pre>	<u>Import datetime</u> The 'datetime' is imported to handle tasks that related to date, such as validate registration date of a customer. <u>File Paths</u> The variables define the paths to different files used to store various types of data in the application. <u>Superuser Credentials</u> The list contains the credentials for the superuser, including the username and password of the superuser. <u>Read Files Function</u> To read the content of a file and return it as a list of lists. <u>Write Files Function</u> To write data to a file <u>Sign Up Function</u> To handle sign up process for different types of users. 1. Prompt user to select their user type (Customer,

<pre> elif nationality == "M": passport_number = input("Enter your passport number: ").strip() phone_number = input("Enter your phone number (either Malaysia phone number or international phone number): ").strip() customer_info = ["Non-Malaysian", passport_number, phone_number] else: print("Invalid nationality!") return registration_date = input("Enter your registration date (YYYY-MM-DD): ").strip() try: datetime.datetime.strptime(registration_date, "%Y-%m-%d") except ValueError: print("Invalid date format! Please use YYYY-MM-DD.") return customer_data = [username, password, "Pending", user_type] + customer_info + [registration_date] login_data.append(customer_data) elif user_type == "2": if not (len(password) >= 10 and password.startswith("ADMIN@")): print("Admin password must be at least 10 characters long and start with 'ADMIN@'.") return login_data.append([username, password, "Pending", user_type]) elif user_type == "3": if not (len(password) >= 8 and password.startswith("INV@") and sum(c.isdigit() for c in password) == 4): print("Inventory staff password must be at least 8 characters long, start with 'INV@' and followed by 4 numbers.") return login_data.append([username, password, "Pending", user_type]) write_file(login_data_file, login_data) print("Registration successful. Waiting for approval.") def sign_up(): user_type = input("Are you signing up as:\n(1) Customer\n(2) Admin\n(3) Inventory Staff\n") if user_type not in ["1", "2", "3"]: print("Invalid selection!") return login_data = read_file(login_data_file) if user_type == "2": username = get_next_admin_username(login_data) print(f"Generated admin username: {username}") else: username = input("Create a username: ") password = input("Create a password: ") if user_type == "1": if not (len(password) >= 8 and any(c.isupper() for c in password) and sum(c.isdigit() for c in password) >= 3): print("Customer password must be at least 8 characters long, including 1 UPPER CASE and 3 NUMBERS.") return # NATIONALITY nationality = input("Are you Malaysian or non-Malaysian? Enter 'M' for Malaysian or 'N' for Non-Malaysian: ").strip() if nationality == "M": ic_number = input("Enter IC number (excluding '-')").strip() if len(ic_number) != 12: print("Invalid IC number format.") return phone_number = input("Enter your phone number (12 characters starting with +60): ").strip() if len(phone_number) != 12 or not phone_number.startswith("+60"): print("Phone number must start with +60 and be 12 characters long.") return customer_info = ["Malaysian", ic_number, phone_number] </pre>	Admin or Inventory Staff). 2. Check if the selection is valid. 3. Read existing login data from the file. <u>User Data Collection</u> To collect additional information based on user type - It will collect nationality, IC number (for Malaysians) or passport number from customers. - It also validates phone numbers and registration date of a customer. <u>Write file and print message</u> To save the updated login data to the file (write the modified "login_data" to "login_data_file" and print a confirmation message to inform the user that their registration is successful and pending approval.
---	--

```

#login
def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    login_data = read_file(login_data_file)
    superuser_credentials = ("superuser@123", "superusernotsuperman")

    if username == superuser_credentials[0] and password == superuser_credentials[1]:
        superuser_menu()
        return

    found = False
    for user in login_data:
        if user[0] == username and user[1] == password:
            found = True
            if user[2] == "Approved":
                if user[3] == "1":
                    customer_menu()
                elif user[3] == "2":
                    log_activity(username, "Logged in")
                    admin_menu()
                elif user[3] == "3":
                    log_activity(username, "Logged in")
                    inventory_menu()
            else:
                print("You do not have access to this feature.")
                break

    if not found:
        print("Invalid login credentials!")
        log_activity(username, "Failed login attempt")

```

User input

To prompt the user to enter their username and password to login.

Read Login Data

To read the login data from the file "login_data_file" and store it in "login_data" variable. The data is used to verify user credentials.

Define Superuser Credentials

To define the fixed credentials for the superuser. It's used to check if the login attempt is from the superuser.

Check for Superuser Login

To check whether the entered username and password match the superuser credentials.

- If it's match, it calls the "superuser_menu()" function
- The "return" statement stops the function, the rest of code for superuser login is skipped

Search for User in Login Data

To check whether the entered username and password match any entry in "login_data".

- Initial "found" as "False".

	- Check each user in “login_data”. - If the credentials is match and the account is “Approved”, it directs the user to appropriate menu based on their user type. - If not approved, it will display a restricted access message. - The loop will stop once a match is found. <u>Handle Invalid Credentials</u> If no match found in the login data, it will print an error message indicate invalid credentials. - Log the failed login attempt with “log_activity(username, “Failed login attempt”).
--	---

```

#Admin Menu
def admin_menu():
    welcome_admin = (
        """
        .....
        Admin Menu\n
        .....
    """
    )
    print(welcome_admin)

    while True:
        print("\n1. Verify New Customer")
        print("2. Check Customer Order Status (Purchase Order)")
        print("3. Repair/Service Order")
        print("4. Purchase order (SUPPLIER)")
        print("5. Reports")
        print("6. Logout")

        choice = input("Enter choice: ")

        if choice == "1":
            verify_new_customer()
        elif choice == "2":
            CUSpurchase_order()
        elif choice == "3":
            repair_service()
        elif choice == "4":
            SUPpurchase_order()
        elif choice == "5":
            reports("2")
        elif choice == "6":
            break
        else:
            print("Invalid choice!")

```

Welcome message

To print a decorative banner for the Admin Menu.

Admin Menu Loop

To display a menu of options for the admin to choose from and repeatedly ask for input

- The “While True:” loop keep the menu running until the admin choose to logout.
- “choice = input(“Enter choice:”)” collect the admin’s choice

Handle Admin Choices

To execute the function based on the admin’s choice

- Choice 1: Call “verify_new_customer()” to verify new customer accounts.
- Choice 2: Call “CUSpurchase_order()” to check the status of customer purchase orders.
- Choice 3: Call “repair_service()” to handle supplier purchase orders.
- Choice 4: Call “SUPpurchase_order()”
- Choice 5: Call “reports(“2”)” to generate admin related reports.

	- Choice 6: Break the loop, logout from admin menu and return to the previous menu (main menu). - Invalid Choice: If the input doesn't match any option, it prints "Invalid choice!" and re-displays the menu.
<pre> #Super user menu def superuser_menu(): welcome_superuser = (""" \n " SUPER USER MENU\n" ) print(welcome_superuser) while True: print("\n1. Add Users") print("2. Modify User Personal Details") print("3. Disable User Access") print("4. Inquiry of User's System Usage") print("5. Check Customer Order Status (Purchase Order)") print("6. Repair/Service Order") print("7. Purchase Order (SUPPLIER)") print("8. Reports") print("9. Logout") choice = input("Enter choice: ") if choice == "1": add_users() elif choice == "2": modify_user_personal_details() elif choice == "3": disable_user_access() elif choice == "4": inquiry_of_user_system_usage() elif choice == "5": CUSpurchase_order() elif choice == "6": repair_service() elif choice == "6": repair_service() elif choice == "7": SUPpurchase_order() elif choice == "8": reports("superuser") elif choice == "9": break else: print("Invalid choice!") </pre>	<u>Welcome message</u> To print a decorative banner for the Superuser Menu. <u>Superuser Menu Loop</u> To display a menu of options for the superuser to choose from and repeatedly ask for input. - The "while true:" loop keep the menu active until the superuser wanted to logout <u>Handle Menu Choices</u> To execute functions based on the superuser's menu choice. - Choice "1": Call "add_users()" to add new users - Choice "2": Call "modify_user_personal_details" to update user details - Choice "3": Call "disable_user_access()" to deactivate a user's access - Choice "4": Call "inquiry_of user s

	system_usage()” to review user activity - Choice “5”: Call “CUSpurchase_order()” to check customer order statuses - Choice “6”: Call “repair_service()” to handle repair or service orders - Choice “7”: Call “SUPpurchase_order()” to check supplier orders statuses - Choice “8”: Call “reports(“superuser”)” to generate reports for the superuser - Choice “9”: Break the loop and logout the superuser - Invalid Choice: If the input doesn’t match any option, it prints “Invalid choice!” and re-displays the menu
<pre>def inventory_menu(): welcome_inventory = (""" INVENTORY MAIN MENU\n """) greet_user(username) print(welcome_inventory) while True: print(reminder_msg, "\n(Example: enter '1' to manage inventory item.)") print("\nPlease select your action:\n" "1. Manage Inventory Item\n" "2. Manage Customer Orders\n" "3. Manage Purchase Orders\n" "4. View Report\n" "5. Logout\n") try: choice = int(input("Your input: ")) if choice == 1: manage_inventory() elif choice == 2: manage_customer_orders() elif choice == 3: manage_purchase_orders() elif choice == 4:</pre>	<u>Welcome message</u> To print a decorative banner for the Customer Menu. <u>Greet User and Print Menu</u> To display a greeting and the menu header. -

<pre> view_report() elif choice == 5: break elif choice not in range(1,6): print("Enter selections displayed only.") print("-----") except ValueError: print("Enter option NUMBER only.") print("-----") </pre>	
<pre> elif choice == "7": reports("1") elif choice == "8": break else: print("Invalid choice!") def customer_menu(): welcome_customer = ("-----" " Welcome to Customer Menu\n" "-----") print(welcome_customer) inventory_file = "inventory_item.txt" # Specify the inventory file name username = "customer" while True: print("\n1. Purchase Order") print("2. Repair/Service Order") print("3. Modify Purchase Order/Repair/Service Order") print("4. Payment for orders placed") print("5. Inquiry of order status") print("6. Cancel Order") print("7. Reports") print("8. Logout") choice = input("Enter choice: ").strip() if choice == "1": purchase_order(username, inventory_file) elif choice == "2": service_repair_order(username) elif choice == "3": modify_order(username, inventory) elif choice == "4": make_payment(username) elif choice == "5": inquire_order_status(username) elif choice == "6": cancel_order(username) </pre>	<u>Welcome message</u> To print a decorative banner for the Customer Menu. <u>Customer Menu Loop</u> To display a menu of options for the customer to choose from and repeatedly ask for input. - The “while true:” loop keep the menu active until the customer wanted to logout - “inventory_file” specified the file used for inventory items <u>Handle Menu Choices</u> - Choice “1”: Call “manage_inventory()” to manage inventory items - Choice “2”: Call “manage_customer_orders” to manage purchase orders - Choice “3”: Call “manage_purchase_orders()” to manage purchase orders - Choice “4”: Call “view_reports()” to view reports - Choice “5”: Break the loop and logout the inventory menu

	- Invalid Choice: If the input doesn't match any option, it prints "Invalid choice!" and re-displays the menu
--	---

Source code

```
def purchase_order(username, inventory_file):
    view_inventory(inventory_file) # Display the inventory

    order_items = []
    total_amount = 0.0

    while True:
        item_id = input("Enter the ItemID to purchase (or type 'done' to finish): ").strip()

        if item_id.lower() == 'done':
            break

        try:
            quantity = int(input("Enter the quantity: ").strip())
        except ValueError:
            print("Invalid quantity. Please enter a number.")
            continue

        item_found = False
        updated_inventory = []

        with open(inventory_file, "r") as file:
            lines = file.readlines()

        for line in lines:
            item = line.strip().split(", ")
            if len(item) != 11:
                continue

            if item[0].strip() == item_id:
                item_name = item[1].strip()
                try:
                    price = float(item[9].strip())
                    stock = int(item[10].strip())
                except ValueError:
                    continue

        def purchase_order(username, inventory_file):
            view_inventory(inventory_file) # Display the inventory

            order_items = []
            total_amount = 0.0

            while True:
                item_id = input("Enter the ItemID to purchase (or type 'done' to finish): ").strip()

                if item_id.lower() == 'done':
                    break

                try:
                    quantity = int(input("Enter the quantity: ").strip())
                except ValueError:
                    print("Invalid quantity. Please enter a number.")
                    continue

                item_found = False
                updated_inventory = []

                with open(inventory_file, "r") as file:
                    lines = file.readlines()

                for line in lines:
                    item = line.strip().split(", ")
                    if len(item) != 11:
                        continue

                    if item[0].strip() == item_id:
                        item_name = item[1].strip()
                        try:
                            price = float(item[9].strip())
                            stock = int(item[10].strip())
                        except ValueError:
                            continue

                    item_found = True

                    if quantity > stock:
                        print(f"Insufficient stock for {item_name}. Only {stock} available.")
                        updated_inventory.append(line.strip())
                        continue

                    order_items.append((item_id, item_name, quantity, price))
                    total_amount += price * quantity
                    item[10] = str(stock - quantity) # Decrease the stock
                    updated_inventory.append(", ".join(item))
                    print(f"Added {quantity} of {item_name} to your order.")
                else:
                    updated_inventory.append(line.strip())

            if not item_found:
                print(f"ItemID {item_id} not found.")

            with open(inventory_file, "w") as file:
                for item_line in updated_inventory:
                    file.write(item_line + "\n")
```

	This function allows the customer to place an order by selecting items and quantities from the inventory. Display Inventory shows the current inventory using view_inventory(inventory_file). order_items: List to keep track of selected items. total_amount: Total cost of the order. Order Input Loop: - Repeatedly prompts for item ID and quantity until 'done' is entered. - Checks for valid number input. Process Inventory: -Reads the inventory file, finds the item, and checks stock. -Updates stock and adds items to the order if sufficient stock is available. - Writes the updated stock levels back to the file. - Prints a summary of ordered items and the total amount. If customer confirmed to make order, saves the order using save_order(username, order_items, total_amount).
Source code	<pre>def save_inventory_to_file(inventory, filename): with open(filename, "w") as file: for item in inventory: # Convert each item list to a string and write it to the file file.write(",".join(map(str, item)) + "\n") inventory = [["ItemID", "Item Name", "Brand", "Category", "Model Number", "Description", "Specification", "Warranty(Year)", "Stock Keeping Unit", "Price/Unit(₹)", "Quantity"], ["101", "Laptop", "Dell", "Laptop", "Inspiron 15 3000", "15.6-inch laptop with Intel i5 processor", "8GB RAM, 256GB SSD", 2, "SKU101", 2300.00, 25], ["102", "Desktop", "HP", "Desktop", "Pavilion 590", "Powerful desktop for everyday use", "16GB RAM, 1TB HDD", 3, "SKU102", 3200.00, 10], ["103", "Monitor", "Samsung", "Monitor", "LC24F396FHUKZA", "Curved 24-inch monitor", "1920x1080 resolution, 60Hz", 2, "SKU103", 650.00, 15], ["104", "Mouse", "Logitech", "Accessory", "M330", "Silent wireless mouse", "Wireless, ergonomic design", 1, "SKU104", 75.00, 50], ["105", "Keyboard", "Corsair", "Accessory", "K95 RGB", "Mechanical gaming keyboard", "RGB backlight, Cherry MX switches", 2, "SKU105", 550.00, 20], ["106", "Printer", "Canon", "Printer", "PIXMA G2020", "High-efficiency color printer", "Ink tank system, USB 2.0", 1, "SKU106", 450.00, 30], ["107", "External Hard Drive", "Seagate", "Storage", "Backup Plus", "Portable external hard drive", "2TB storage, USB 3.0", 2, "SKU107", 350.00, 40], ["108", "SSD", "Samsung", "Storage", "970 EVO", "High-speed SSD for gaming", "1TB NVM, M.2", 5, "SKU108", 750.00, 25], ["109", "Graphics Card", "NVIDIA", "Component", "RTX 3070", "Powerful GPU for gaming", "8GB GDDR6, Ray tracing support", 3, "SKU109", 2500.00, 15], ["110", "Power Supply", "Corsair", "Component", "RM850x", "High-performance power supply", "850W, 80+ Gold certified", 3, "SKU110", 450.00, 35], ["111", "Motherboard", "ASUS", "Component", "ROG Strix Z490-E", "Gaming motherboard", "LGA 1200, ATX, DDR4 support", 3, "SKU111", 900.00, 10], ["112", "RAM", "Kingston", "Component", "HyperX Fury", "High-speed RAM for gaming", "16GB DDR4, 3200MHz", 5, "SKU112", 350.00, 60], ["113", "CPU", "Intel", "Component", "Core i7-10700K", "High-performance CPU", "8 cores, 16 threads, 3.8GHz", 3, "SKU113", 1500.00, 20], ["114", "Laptop Charger", "Dell", "Accessory", "LA65HS2-01", "65W power adapter for laptops", "65W, 19.5V", 2, "SKU114", 150.00, 50], ["115", "Webcam", "Logitech", "Accessory", "C920 HD Pro", "Full HD webcam", "1080p, 30fps", 2, "SKU115", 350.00, 20], ["116", "Wireless Router", "TP-Link", "Networking", "Archer AX6000", "High-speed Wi-Fi 6 router", "8-stream, 1.8GHz quad-core CPU", 3, "SKU116", 600.00, 15], ["117", "Laptop Cooling Pad", "Cooler Master", "Accessory", "Notepal X3", "Cooling pad for laptops", "120mm fan, adjustable height", 2, "SKU117", 120.00, 40], ["118", "USB Flash Drive", "SanDisk", "Storage", "Ultra Flair", "High-speed USB flash drive", "128GB, USB 3.0", 1, "SKU118", 65.00, 75], ["119", "Headset", "Razer", "Accessory", "Kraken X", "Wired gaming headset", "7.1 surround sound, lightweight design", 2, "SKU119", 250.00, 30], ["120", "Gaming Chair", "Secretlab", "Furniture", "Omega 2020", "Comfortable gaming chair", "Adjustable armrests, lumbar support", 5, "SKU120", 1200.00, 10]] save_inventory_to_file(inventory, "inventory_item.txt")</pre> The missing word is Quantity In Stock
Explanation	The save_inventory_to_file function takes a list of items (inventory) that our store sells and saves it into a text file called inventory_item.txt.

Source code	<pre> def view_inventory(filename): with open(filename, "r") as file: lines = file.readlines() headers = lines[0].strip().split(", ") print(f'{' '.join(headers)}') print("-" * 100) for line in lines[1:]: item = line.strip().split(", ") if len(item) != len(headers): print(f"Skipping malformed line: {line.strip()}") continue print(f"ItemID: {item[0]}") print(f"Item Name: {item[1]}") print(f"Brand: {item[2]}") print(f"Category: {item[3]}") print(f"Model Number: {item[4]}") print(f>Description: {item[5]}") print(f"Specification: {item[6]}") print(f"Warranty (Year): {item[7]}") print(f"Stock Keeping Unit: {item[8]}") print(f"Price/Unit (RM): {item[9]}") print(f"Quantity In Stock: {item[10]}") print("-" * 100) </pre>
Explanation	Headers and Formatting: The first line of the file (which contains the headers) is stripped of any extra whitespace, split by commas, and stored in the headers list. The headers are then printed in a horizontal format, separated by " ", followed by a separator line for clarity. Item Display: This function iterates over each line in lines, starting from the second line (since the first line contains headers). Each line is stripped of extra whitespace, split by commas, and stored in the item list. If a line doesn't match the expected number of fields, the function skips it and prints a message indicating the issue. The function then prints each attribute of the item on a new line, using descriptive labels like ItemID, Item Name, Brand, etc. Separator Line: After displaying all the attributes of an item, the function prints a separator line to visually separate each item for readability.

Source code

```
def service_repair_order(username):
    service_options = display_service_options()
    repair_options = display_repair_options()
    service_name = ""
    repair_name = ""

    print("Service/Repair Order Menu")
    print("1. Service")
    print("2. Repair")

    choice = input("Choose an option (1 for Service, 2 for Repair): ")

    if choice == '1':
        print("Available Service Options:")
        for key, value in service_options.items():
            print(f"{key}. {value['name']}")

        service_choice = input("Select a service option (1-4): ")

        if service_choice in service_options:
            print(f"--- {service_options[service_choice]['name']} ---")
            display_options(service_options[service_choice]['options'])
            specific_choice = input("Select a specific service option (a-h): ")

            if specific_choice in service_options[service_choice]['options']:
                service_name = service_options[service_choice]['name']
                option_details = service_options[service_choice]['options'][specific_choice]
                amount = int(option_details.split('RM')[1].strip())
            else:
                print("Invalid choice. Returning to customer menu.")
                return
        elif choice == '2':
            print("Available Repair Options:")
            for key, value in repair_options.items():
                print(f"{key}. {value['name']}")

            repair_choice = input("Select a repair option (1-2): ")

            if repair_choice in repair_options:
                print(f"--- {repair_options[repair_choice]['name']} ---")
                display_options(repair_options[repair_choice]['options'])
                specific_choice = input("Select a specific repair option (a-k): ")

                if specific_choice in repair_options[repair_choice]['options']:
                    repair_name = repair_options[repair_choice]['name']
                    option_details = repair_options[repair_choice]['options'][specific_choice]
                    amount = int(option_details.split('RM')[1].strip())
                else:
                    print("Invalid choice. Returning to customer menu.")
                    return
            else:
                print("Invalid choice. Returning to customer menu.")
                return
        else:
            print("Invalid choice. Returning to customer menu.")
            return

    print("Please select your Device Transfer Method:")
    print("1. Drop off")
    print("2. Pickup (additional RM30)")

    transfer_method = input("Enter your choice (1 or 2): ")
    additional_charge = 0

    if transfer_method == '1':
        print("Business hours: Mon-Fri 9AM - 6PM")
    elif transfer_method == '2':
        additional_charge = 30
        pickup_address = input("Enter pickup address: ")
        pickup_time = input("Enter pickup time: ")
    else:
        print("Invalid choice. Returning to customer menu.")
        return

    total_amount = amount + additional_charge
    print(f"Total Amount (including additional charges): RM{total_amount:.2f}")
```

	<pre> confirm = input("Are you sure you want to place the order? (yes/no): ") if confirm.lower() == 'yes': order_id = generate_order_id() order_date = datetime.datetime.now().strftime('%Y-%m-%d') order_data = { "username": username, "order_id": order_id, "order_date": order_date, "service_or_repair": {"type": "Service" if choice == '1' else "Repair"}, "service_or_repair_name": service_name if choice == '1' else repair_name, "selected_option": option_details, "total_amount": f"RM{total_amount:.2f}", "transfer_method": "Drop off" if transfer_method == '1' else "Pickup", "status": "Unpaid" } if transfer_method == '2': order_data["pickup_address"] = pickup_address order_data["pickup_time"] = pickup_time try: with open("orders.json", "r+") as file: try: orders = json.load(file) except json.JSONDecodeError as e: print(f"Error loading JSON data: {e}") orders = [] orders.append(order_data) file.seek(0) json.dump(orders, file, indent=4) file.truncate() print(f"Order placed successfully! Your order ID is {order_id}.") print("Please proceed to make payment.") except IOError: print("Error saving order. Please try again later.") else: print("Order cancelled.") </pre>
Explanation	Allows the customer to place a service or repair order, including selecting service or repair options, choosing a device transfer method, and confirming the order. • Prompts the user to choose between service or repair. • Prompts for service selection and specific service choice. • Displays available service and repair options. • Prompts for the transfer method: Drop off or Pickup. • Drop off: No additional charge. • Pickup: Adds RM30 to the total amount and prompts for pickup address and time. • Calculates the total amount including any additional charges and displays the total amount to the customer. • Prompts the customer to confirm the order. • If confirmed, generates an order ID and captures the current date. Saves the order data to orders.json, handling potential JSON and IO errors. • If canceled, prints an order cancellation message.

Source code

```
def display_service_options():
    service_options = {
        "1": {
            "name": "Hardware Services",
            "options": {
                "a": "Memory RAM (2GB) RM100",
                "b": "Memory RAM (4GB) RM200",
                "c": "Memory RAM (8GB) RM300",
                "d": "Computer performance upgrade RM300",
                "e": "HDD (500GB) RM150",
                "f": "HDD (1TB) RM250",
                "g": "SSD (500GB) RM250",
                "h": "SSD (1TB) RM450",
            }
        },
        "2": {
            "name": "Software Services",
            "options": {
                "a": "Computer cleaning service: RM100",
                "b": "Operating system updates: RM150",
                "c": "Operating system installation: RM150",
                "d": "Window/OS upgrade: RM150",
                "e": "Backup and Data Recovery: RM200"
            }
        },
        "3": {
            "name": "IT Services and Support",
            "options": {
                "a": "Battery: RM200",
                "b": "IT product technical support: RM300",
                "c": "IT maintenance: RM300",
                "d": "Hard Disk (500GB): RM200",
                "e": "Hard Disk (1TB): RM300",
                "f": "Solid State Drive (500GB): RM300",
                "g": "Solid State Drive (1TB): RM500",
                "h": "LED / LCD display screen: RM500"
            }
        },
        "4": {
            "name": "Networking Services",
            "options": {
                "a": "Wi-Fi/LAN networking setup: RM100",
                "b": "Wi-Fi/LAN networking upgrades: RM100",
                "c": "Wireless setup or installation: RM150",
                "d": "Extend internet/Wi-Fi coverage: RM300"
            }
        }
    }
    return service_options

def display_repair_options():
    repair_options = {
        "1": {
            "name": "Hardware Services",
            "options": {
                "a": "PC start-up issue: RM100",
                "b": "Disk drive defragmentation service: RM100",
                "c": "Charging problem repair: RM100",
                "d": "Keyboard replacement: RM200",
                "e": "Battery replacement: RM200",
                "f": "Tablet screen replacement: RM200",
                "g": "HDD/SSD installation: RM300",
                "h": "Laptop screen repair: RM300",
                "i": "Water or liquid damage repair: RM300",
                "j": "Fix graphic card: RM300",
                "k": "Fix damaged motherboard: RM300"
            }
        },
        "2": {
            "name": "Software Services",
            "options": {
                "a": "Basic maintenance: RM100",
                "b": "Anti-virus installation: RM100",
                "c": "Operating System Updates: RM150",
                "d": "Malware removal: RM150",
                "e": "Remove trojans and spyware: RM150",
                "f": "Fix networking issues: RM150",
                "g": "Diagnose software issues: RM150",
                "h": "Reformat computer: RM200",
                "i": "Virus removal: RM200",
                "j": "Data recovery: RM300",
            }
        }
    }
    return repair_options
```



```

while True:
    print("\n1. Modify Item ID")
    print("2. Modify Quantity")
    print("3. Modify Service/Repair")
    print("4. Done")
    modify_choice = input("Enter your choice: ").strip()

    if modify_choice == "1":
        # Modify Item ID
        item_id = input("Which ItemID do you want to modify: ").strip()
        new_item_id = input("Enter the new ItemID: ").strip()
        new_quantity = input("Enter the new quantity: ").strip()
        for i, line in enumerate(order):
            if line.startswith("ItemID: " + item_id):
                item_details = line.split(", ")
                item_details[0] = f"ItemID: {new_item_id}"
                item_details[2] = f"Quantity: {new_quantity}"
                order[i] = ", ".join(item_details)
                break
        print("Item ID modified successfully.")

    elif modify_choice == "2":
        # Modify Quantity
        item_id = input("Enter the ItemID to modify quantity: ").strip()
        new_quantity = input("Enter the new quantity: ").strip()
        for i, line in enumerate(order):
            if line.startswith("ItemID: " + item_id):
                item_details = line.split(", ")
                item_details[2] = f"Quantity: {new_quantity}"
                order[i] = ", ".join(item_details)
                break
        print("Item quantity modified successfully.")

    elif modify_choice == "3":
        # Modify Service/Repair Order
        print("\n1. Modify Service")
        print("2. Modify Repair")
        print("3. Modify Device Transfer Method")
        service_choice = input("Enter your choice: ").strip()

        if service_choice == "1":
            # Modify Service
            service_options = display_service_options()
            print("Available Service Options:")
            for key, value in service_options.items():
                print(f"{key}: {value['name']}")
            selected_service = input("Select a service option (1-4): ").strip()
            if selected_service in service_options:
                print("Specific Options:")
                for option_key, option_value in service_options[selected_service]['options'].items():
                    print(f"{option_key}: {option_value}")
                specific_choice = input("Select a specific service option (a-h): ").strip()
                if specific_choice in service_options[selected_service]['options']:
                    service_name = service_options[selected_service]['name']
                    option_details = service_options[selected_service]['options'][specific_choice]
                    amount = float(option_details.split('RM')[1].strip())
                    for i, line in enumerate(order):
                        if line.startswith("Service/Repair Option: "):
                            order[i] = f"Service/Repair Option: {service_name}"
                        elif line.startswith("Specific Option: "):
                            order[i] = f"Specific Option: {option_details}"
                        elif line.startswith("Total Amount: "):
                            order[i] = f"Total Amount: RM{amount:.2f}"
                    print("Service order modified successfully.")
            else:
                print("Invalid service option.")

```

	<pre> elif service_choice == "2": # Modify Repair repair_options = display_repair_options() print("Available Repair Options:") for key, value in repair_options.items(): print(f"{key}: {value['name']}") selected_repair = input("Select a repair option (1-2): ").strip() if selected_repair in repair_options: print("Specific Options:") for option_key, option_value in repair_options[selected_repair]['options'].items(): print(f"{option_key}: {option_value}") specific_choice = input("Select a specific repair option (a-k): ").strip() if specific_choice in repair_options[selected_repair]['options']: repair_name = repair_options[selected_repair]['name'] option_details = repair_options[selected_repair]['options'][specific_choice] amount = float(option_details.split('RM')[1].strip()) for i, line in enumerate(order): if line.startswith("Service/Repair Option: "): order[i] = f"Service/Repair Option: {repair_name}" elif line.startswith("Specific Option: "): order[i] = f"Specific Option: {option_details}" elif line.startswith("Total Amount: "): order[i] = f"Total Amount: RM{amount:.2f}" print("Repair order modified successfully.") else: print("Invalid repair option.") else: print("Invalid choice. Please try again.") if not order_found: print("Order ID not found or already paid. Please make an order first.") return except FileNotFoundError: print("Order file not found. Please make an order first.") elif service_choice == "3": print("\n1. Drop Off") print("\n2. Pick Up") transfer_method = input("Enter your choice: ").strip() if transfer_method == "1": for i, line in enumerate(order): if line.startswith("Device Transfer Method: "): order[i] = f"Device Transfer Method: Drop Off" print("Device transfer method modified to Drop Off.") elif transfer_method == "2": additional_charge = 30 pickup_address = input("Enter pickup address: ") pickup_time = input("Enter pickup time: ") for i, line in enumerate(order): if line.startswith("Device Transfer Method: "): order[i] = f"Device Transfer Method: Pick Up (Address: {pickup_address}, Time: {pickup_time}, Additional Charge: RM{additional_charge})" print("Device transfer method modified to Pick Up.") else: print("Invalid choice. Returning to customer menu.") break elif modify_choice.lower() == "4": # Save changes to the file with open("save_order.txt", "w") as file: for o in orders: for line in o: file.write(line + "\n") file.write("-" * 40 + "\n") print("Purchase order modified successfully. Returning to customer menu.") customer_menu() # Call without parameters if globals are used return </pre>
Explanation	This function allows a user to modify an existing order. Ask the user to enter the Order ID they wish to modify. Read and store the order details from the file save_order.txt. Read all orders from the file and find the one matching the entered Order ID and username. Display the details of the found order to the user. Modification Menu: Provide options to modify: Item ID: Change the Item ID and quantity. Quantity: Update the quantity of a specific item. Service/Repair: Modify the type of service or repair requested, including: -Service Options: Choose and update a service from available options. -Repair Options: Choose and update a repair from available options. -Device Transfer Method: Choose between Drop Off or Pick Up and provide additional details if needed.

	Save all modifications back to the save_order.txt file. Confirm that the changes have been successfully saved and return to the customer menu. If the order ID is not found or the file does not exist, inform the user accordingly.
--	---

Source code	<pre>def make_payment(username): order_id = input("Enter the Order ID for payment: ").strip() try: with open('orders.json', 'r') as file: orders = json.load(file) order_found = False for order in orders: if order["username"] == username and order["order_id"] == order_id and order.get("status", "") == "Unpaid": order_found = True payment_amount = float(input("Enter payment amount: ").strip()) if payment_amount <= 0: print("Invalid payment amount. Returning to menu.") return if payment_amount != order.get("total_amount", 0): print("Payment amount does not match the order total. Payment not processed.") return order["status"] = "Paid" print(f"Payment processed successfully for Order ID {order_id}.") # Generate the invoice after payment is processed generate_invoice(order) break if not order_found: print("Order ID not found or already paid. Payment not allowed.") return with open('orders.json', 'w') as file: json.dump(orders, file, indent=4) except IOError: print("Error accessing orders file.")</pre>
	This function is designed to handle payment processing for an order. The user is asked to enter the Order ID for which they want to make a payment. Access Orders Data Read Orders File: Load existing orders data from the orders.json file. Search for Order: Iterate through the orders to find a match with the entered Order ID and ensure it is associated with the given username and is marked as "Unpaid". Process Payment: Validate Payment Amount: Check if the entered payment amount is valid (greater than 0) and matches the order's total amount. Update Order Status: If the payment is correct, update the order status to "Paid" and print a success message. Generate Invoice: After processing the payment, generate an invoice for the order. Handle Errors -If the order is not found or has already been paid, display an appropriate message. -Handle any errors related to accessing the orders.json file. Save Changes -Save the updated orders data back to the orders.json file.

Source code

```
def inquire_order_status(username):
    order_summary = {
        'To Pay': [],
        'Packing': [],
        'Shipping': [],
        'Delivered': [],
        'Cancelled': []
    }

    # Check if the order files exist
    if not os.path.exists('save_order.txt'):
        print("No orders found.")
        return

    try:
        with open('save_order.txt', 'r') as file:
            lines = file.readlines()
    except IOError:
        print("Error reading orders file.")
        return

    # Initialize variables to store the current order information
    current_order = {}

    for line in lines:
        line = line.strip()
        if not line:
            continue
        print(f"Processing line: '{line}'")

        if line.startswith('Order ID:'):
            current_order['order_id'] = line.split(':')[1]
        elif line.startswith('Order by:'):
            current_order['username'] = line.split(':')[1]
        elif line.startswith('Status:'):
            current_order['status'] = line.split(':')[1]
        elif line.startswith('Total Amount:'):
            current_order['total_amount'] = line.split(':')[1]
        elif line.startswith('-----'):
            # End of order, process it
            if current_order.get('username') == username:
                order_id = current_order.get('order_id')
                status = current_order.get('status')

                if status == 'Unpaid':
                    order_summary['To Pay'].append(order_id)
                elif status == 'Paid':
                    try:
                        order_date_str = current_order.get('order_id')[-8:]
                        order_date = datetime.strptime(order_date_str, '%Y%m%d')
                    except ValueError:
                        order_date = datetime.now()

                    order_age = (datetime.now() - order_date).days
                    if order_age <= 1:
                        order_summary['Packing'].append(order_id)
                    elif order_age == 2:
                        order_summary['Shipping'].append(order_id)
                    elif order_age > 3:
                        order_summary['Delivered'].append(order_id)
                elif status == 'Cancelled':
                    order_summary['Cancelled'].append(order_id)

            current_order = {}

    print("Order Status Summary:")
    for category, orders in order_summary.items():
        if orders:
            print(f"{category}: {' '.join(orders)}")
        else:
            print(f"{category}: -")
```

Explanation	This function is designed to inquire about the status of orders for user Create a dictionary order_summary with categories ('To Pay', 'Packing', 'Shipping', 'Delivered', 'Cancelled') to hold order IDs for each status. Check for Order Files: -Verify if the save_order.txt file exists. If not, print a message indicating no orders are found and return. -Attempt to read the orders from save_order.txt. If there's an error reading the file, print an error message and return. Process Orders: -Use current_order to temporarily store information about the current order being processed. -Read each line from the file: • Order ID: Capture the order ID. • Username: Capture the username associated with the order. • Status: Capture the current status of the order. • Total Amount: Capture the total amount of the order. • End of Order: When encountering the end-of-order marker ('----- -----'): -Check User: If the order belongs to the given username, categorize it based on its status: Unpaid: Add the order ID to the 'To Pay' category. Paid: Determine the order age and categorize it into 'Packing', 'Shipping', or 'Delivered' based on the order's age. Cancelled: Add the order ID to the 'Cancelled' category. -Clear “current_order” for the next order. The function categorizes and organizes orders based on their status and prints the summary for each category.
-------------	---

Source code	<pre> #Functions for superuser and admin def add_users(): print("Add users") login_data = read_file(login_data_file) user_type = input("Enter user type (1 for Customer 2 for Admin 3 for Inventory Staff): ").strip() if user_type == "2": # Generate a username for Admin username = get_next_admin_username(login_data) print(f"Generated admin username: {username}") else: username = input("Enter username: ").strip() password = input("Enter password: ").strip() if user_type == "1": # Customer if not (len(password) >= 8 and any(c.isupper() for c in password) and sum(c.isdigit() for c in password) >= 3): print("Customer password must be at least 8 characters long, including 1 UPPER CASE and 3 NUMBERS.") return elif user_type == "2": # Admin if not (len(password) >= 10 and password.startswith("ADMIN@")): print("Admin password must be at least 10 characters long and start with 'ADMIN@'.") return elif user_type == "3": # Inventory Staff if not (len(password) >= 8 and password.startswith("INV@") and sum(c.isdigit() for c in password) == 4): print("Inventory staff password must be at least 8 characters long, start with 'INV@' and followed by 4 numbers.") return # Automatically approve users added by superuser login_data.append([username, password, "Approved", user_type]) write_file(login_data_file, login_data) print("User added successfully.") </pre>
Explanation	<u>Initial Setup and Input</u> To inform the user that they are in the “Add users” section, load existing user data from “login_data_file” and prompt the user to enter the user type they want to add (Customer, Admin or Inventory Staff). <u>Username Generation</u> - If user type is Admin (“2”), the system will automatically generate a unique username for Admin by using “get_next_admin_username()” - Other user type (Customer or Inventory Staff), it prompts the user to enter a username. <u>Password Input and Validation</u> - To prompt the user to enter a password for the new user - To validate the password according to specific criteria based on the user type <u>Adding the user to the system</u> - Appends the new user’s data (username, password, status as “Approved”, and user type) to “login_data”. - The status “Approved” indicates that the user has been granted access immediately. - Updated user data will be saved back to the “login_data_file”. A confirmation message will be display once added successfully.

Source code	<pre> def modify_user_personal_details(): print("Modify User Personal Details") login_data = read_file(login_data_file) username = input("Enter the username of the user to modify: ").strip() for user in login_data: if user[0] == username: print("Current details:", user) while True: print("\nWhat do you want to change?") print("1. Username") print("2. Password") print("3. Approval Status") print("4. User Type") print("5. Nationality (Customers only)") print("6. IC/Passport Number (Customers only)") print("7. Phone Number (Customers only)") print("8. Registration Date") print("9. Exit") choice = input("Enter your choice: ").strip() if choice == "1": new_username = input("Enter new username: ").strip() user[0] = new_username if new_username else user[0] elif choice == "2": new_password = input("Enter new password: ").strip() user[1] = new_password if new_password else user[1] elif choice == "3": new_status = input("Enter new approval status (Approved/Pending): ").strip() user[2] = new_status if new_status else user[2] elif choice == "4": new_user_type = input("Enter new user type (1 for Customer, 2 for Admin, 3 for Inventory Staff): ").strip() user[3] = new_user_type if new_user_type else user[3] elif choice == "5" and user[3] == "1": new_nationality = input("Enter new nationality: ").strip() user[4] = new_nationality if new_nationality else user[4] elif choice == "6" and user[3] == "1": new_ic_passport = input("Enter new IC/Passport Number: ").strip() user[5] = new_ic_passport if new_ic_passport else user[5] elif choice == "7" and user[3] == "1": new_phone_number = input("Enter new phone number: ").strip() user[6] = new_phone_number if new_phone_number else user[6] elif choice == "8": new_registration_date = input("Enter new registration date: ").strip() user[7] = new_registration_date if new_registration_date else user[7] elif choice == "9": break else: print("Invalid choice or not applicable to this user type, please try again.") write_file(login_data_file, login_data) print("User details updated successfully.") return print("User not found.") </pre>
	<pre> def disable_user_access(): print("Disable User Access") login_data = read_file(login_data_file) username = input("Enter the username of the user to disable: ").strip() user_found = False updated_login_data = [] for user in login_data: if user[0] == username: user_found = True print(f"Disabling user: {user[0]}") else: updated_login_data.append(user) if user_found: write_file(login_data_file, updated_login_data) print("User access disabled and user removed from the file successfully.") else: print("User not found.") </pre>
Explanation	<u>Initial Setup and Input</u> - Announce the section's purpose ("Modify User Personal Details") - Load existing user data from "login_data_file". - Prompt the user to enter the username of the user whose details they want to modify <u>Searching for the User</u>

	- Iterate over each user in “login_data” to find a match with the entered username - If a match is found, then it will displays the current details of the user <u>Modify User Details</u> To provide a menu for the user to select which detail to modify. The user can choose: 1. Username 2. Password 3. Approval Status 4. User Type 5. Nationality (Customer) 6. IC/Passport Number(Customer) 7. Phone Number (Customer) 8. Registration Date (Customer) 9. Exit <u>Save changes</u> - After making the necessary modifications, “login_data_file” will saved back the updated user data. <u>User Not Found Case</u> If the loop finish and not matching username is found, it will display “User not found” message.
Source code	<pre>def inquiry_of_user_system_usage(): print("Inquiry of User's System Usage") user_activity_log = read_file(user_activity_log_file) username = input("Enter the username to inquire about: ").strip() found = False for log in user_activity_log: if log[0] == username: print("Activity:", log[1:]) found = True if not found: print("No activity found for this user.") else: print("Inquiry completed.")</pre>
Explanation	<u>Initial Setup and Input</u> - Announce the section’s purpose (“Modify User Personal Details”) - Load existing user data from “login_data_file”. - Prompt the user to enter the username of the user whose details they want to modify <u>Searching for the Userz</u> - Iterate over each user in “login_data” to find a match with the entered username - If a match is found, then it will displays the current details of the user

	<u>Modify User Details</u> To provide a menu for the user to select which detail to modify. The user can choose: 10. Username 11. Pssword 12. Approval Status 13. User Type 14. Nationality (Customer) 15. IC/Passport Number(Customer) 16. Phone Number (Customer) 17. Registration Date (Customer) 18. Exit <u>Save changes</u> - After making the necessary modifications, “login_data_file” will saved back the updated user data. <u>User Not Found Case</u> If the loop finish and not matching username is found, it will display “User not found” message.
--	--

Source code	<pre>def cancel_order(username): order_id = input("Enter the Order ID to cancel: ").strip() try: with open('orders.json', 'r') as file: orders = json.load(file) found = False updated_orders = [] for order in orders: if order['order_id'] == order_id and order.get('status') == 'Unpaid': found = True print(f"Order with ID {order_id} canceled successfully.") else: updated_orders.append(order) if not found: print(f"No un-paid order found with ID {order_id}.") # Write the updated orders back to the JSON file with open('orders.json', 'w') as file: json.dump(updated_orders, file, indent=4) except IOError: print("Error accessing orders file.")</pre>
Explanation	To cancel an unpaid order based on the provided order ID and update the orders record accordingly. • User Input: Prompt the user to enter the Order ID they wish to cancel. Process Orders

	1. Read Existing Orders: - Open the orders.json file for reading and load its contents into the orders list. 2. Initialize Variables: - Set found to False to track if the order to cancel is located. - Create an empty list updated_orders to store orders that will be retained. 3. Iterate Through Orders: Loop through each order in the orders list Check Order ID and Status: ▪ If the order's ID matches the input and its status is 'Unpaid', set found to True, and print a success message indicating the order has been canceled. ▪ If the order does not match the criteria, add it to updated_orders. 4. Handle Non-Existence Case: - If found remains False after processing all orders, print a message stating that no unpaid order with the provided ID was found. Update Orders File • Write Updated Orders: ○ Open the orders.json file for writing. ○ Save the updated_orders list back to the file in JSON format with an indentation for readability. Error Handling • File Access Error: ○ If there's an issue accessing the orders.json file (e.g., file not found or read/write errors), catch the IOError and print an error message.
--	---

Source code	<pre> def verify_new_customer(): print("Verify New Customer") login_data = read_file(login_data_file) # Filter and display only pending customers pending_customers = [user for user in login_data if user[3] == "1" and user[2] == "Pending"] if not pending_customers: print("No pending customers to verify.") return print("Pending Customers:") for user in pending_customers: print(f"Username: {user[0]}") # Ask for the username to verify username_to_verify = input("Enter the username of the customer to approve: ").strip() for user in login_data: if user[0] == username_to_verify and user[3] == "1" and user[2] == "Pending": user[2] = "Approved" # Update status to "Approved" write_file(login_data_file, login_data) print(f"Customer '{username_to_verify}' has been approved.") return print("Customer not found or already approved.") </pre>
-------------	---

Explanation	<u>Announcing the Operation</u> - It reads the content of the “login_data_file” into the “login_data” variable. <u>Filtering Pending Customers</u> - The function uses to filter out customers who are pending approval. - user[3] == ‘1’: To check if the user is a customer (user type “1”) - user[2] == ‘pending’: To check whether the user’s account status is “Pending”. <u>Handling No Pending Customers</u> - If there are no customers in the “Pending” state, it will inform the user by display “No pending customers to verify” <u>Display Pending Customers</u> - If there are any pending customers, it will display a list of their usernames <u>Approve a Customer</u> - Prompt admin to input the customer’s username they want to approve - The “.strip()” is used to remove any leading whitespace from the input - Loop “login_data” to find the matching customer. - user[0] == username_to_verify: The username matches the one entered by the admin - user[3] == 1: The user is a customer - user[2] == Pending: The user’s account still in “Pending” status - If the conditions are met, the status is updated to “Approved”, a confirmation message is printed. <u>Handling Unsuccessful Verification</u> If the loop completes without finding a matching customer, it will display “Customer not found or already approved.”
-------------	---

Source code	<pre> def CUSpurchase_order(): print("Check customer order status") customer_data = read_file(customer_data_file) # Display all orders and their statuses for order in customer_data: print(f"Order ID: {order[0]}, Status: {order[4]}") # Check status of a specific order order_id = input("Enter Order ID to check status: ").strip() # Search for the specified order for order in customer_data: if order[0] == order_id: print(f"Order ID: {order[0]}, Status: {order[4]}") break else: print("Order ID not found.") </pre>
Explanation	<u>Announcing the Operation</u> - It reads the content of the “customer_data_file” into the “customer_data” variable. <u>Display All Orders and Their Statuses</u> - Loop each order in “customer_data” - For each order, it prints the “Order ID” and its “Status” <u>Prompt for an Order ID</u> - To prompt user to enter the “order ID” of the specific order they want to check <u>Searching for the Specified Order</u> - Loop over each order in “customer-data” to find the one that matches the “Order ID” entered by the user - If the order is found, it prints the Order ID and Status of the matching order and exit the loop If the loop completes without finding a matching order, the function will display “Order ID not found”

Source code	<pre> def repair_service(): print("Repair/Service Order") repair_data = read_file(repair_data_file) # Display all repair/service orders and their statuses for repair in repair_data: print(f"Repair ID: {repair[0]}, Status: {repair[5]}") # Check status of a specific repair/service order repair_id = input("Enter Repair ID to check status: ").strip() # Search for the specified repair ID for repair in repair_data: if repair[0] == repair_id: print(f"Repair ID: {repair[0]}, Status: {repair[5]}") break else: print("Repair ID not found.") </pre>
Explanation	<u>Announcing the Operation</u> - It reads the content of the “repair_data_file” into the “repair_data” variable. <u>Display All Repair/Service Order and Their Statuses</u> - Loop each order in “repair_data” - For each order, it prints the “Repair ID” and its “Status” <u>Prompt for a Repair ID</u> - To prompt user to enter the “Repair ID” of the specific order they want to check <u>Searching for the Specified Repair ID</u> - Loop over each order in “repair_data” to find the one that matches the “Repair ID” entered by the user - If the order is found, it prints the Repair ID and Status of the matching order and exit the loop - If the loop completes without finding a matching order, the function will display “Order ID not found”

Source code	<pre> def SUPpurchase_order(): print("Purchase Order (SUPPLIER)") supplier_order_data = read_file(supplier_order_data_file) # Display all supplier orders and their statuses for order in supplier_order_data: print(f"Order ID: {order[0]}, Status: {order[2]}") # Check status of a specific supplier order order_id = input("Enter Order ID to check status: ").strip() # Search for the specified order ID for order in supplier_order_data: if order[0] == order_id: print(f"Order ID: {order[0]}, Status: {order[2]}") break else: print("Order ID not found.") </pre>
Explanation	<u>Announcing the Operation</u> - It reads the content of the “supplier_order_data_file” into the “supplier_order_data” variable. <u>Display All Supplier Orders and Their Statuses</u> - Loop each order in “supplier_order_data” - For each order, it prints the “Order ID” and its “Status” <u>Prompt for an Order ID</u> - To prompt user to enter the “Order ID” of the specific order they want to check <u>Searching for the Specified Order ID</u> - Loop over each order in “supplier_order_data” to find the one that matches the “Order ID” entered by the user - If the order is found, it prints the Order ID and Status of the matching order and exit the loop - If the loop completes without finding a matching order, the function will display “Order ID not found”

<pre>def reports(user_type): print("Reports Menu") if user_type == "superuser": print("1. Invoice") print("2. Financial Reports") print("3. Inventory Reports") print("4. Attendance Reports") print("5. Log of Activity Report") elif user_type == "2": # Admin print("1. Invoice") print("2. Financial Reports") print("3. Inventory Reports") elif user_type == "3": # Inventory Staff print("1. Invoice") print("2. Inventory Reports") elif user_type == "1": # Customer print("1. Invoice") choice = input("Enter the number of the report you want to view: ").strip() if choice == "1": invoice_report() elif choice == "2" and user_type in ["superuser", "2"]: financial_report() elif choice == "3" and user_type in ["superuser", "2", "3"]: inventory_report() elif choice == "4" and user_type == "superuser": attendance_report() elif choice == "5" and user_type == "superuser": log_activity_report() else: print("Invalid choice or unauthorized report.")</pre>	The ‘reports’ function displays different reports based on user type. Superuser can access all report types; Admins can view financial and inventory reports; Inventory Staff can see inventory reports and Customers can only view invoice reports.
<pre>def financial_report(): print("Generating Financial Report...") # Sample financial data financial_data = { "Total Revenue": "RM10,000.00", "Total Expenses": "RM7,500.00", "Net Profit": "RM2,500.00" } print("\nFinancial Report") print("-----") for key, value in financial_data.items(): print(f"{key}: {value}") print("-----\n")</pre>	<u>Announcing the Operation</u> - Print a message indicating the system is generating the financial report. <u>Sample Financial Data</u> - A dictionary named “financial_data” is defined - “Total Revenue” - “Total Expenses” - “Net Profit” <u>Display the Financial Report</u> - Header and divider print “Financial

	Reports” as a title, followed by a line of dashes for formatting - Loop the “financial_data” dictionary using a “for” loop Footer divider prints another line of dashes at the end to close the report.
<pre> ✓ def invoice_report(): print("Generating Invoice Report...") invoice_data = read_file("invoices_data.txt") ✓ if not invoice_data: print("No invoices found.") return print("\nInvoice Report") print("-----") total_amount = 0 # To calculate the total amount ✓ for record in invoice_data: invoice_id = record[0] customer_name = record[1] date = record[2] amount = float(record[3]) # Convert the amount to a floating-point number print(f"Invoice ID: {invoice_id}, Customer: {customer_name}, Date: {date}, Amount: RM {amount:.2f}") total_amount += amount # Add the amount to the total print("-----") print(f"Total Amount: RM {total_amount:.2f}") print("-----\n") </pre>	<u>Reading Invoice Data</u> - Read invoice records from “invoices_data.txt”, If no data is found, it will print “No invoice found” and exits <u>Generating Report</u> - Print a header for the report - Initialize “total_amount” to 0 to calculate the total of all invoice <u>Processing Each Invoice</u> - For each invoice record, it extracts the “invoice_id”, “customer_name”, “date” and “amount”. <u>Display the Total</u>

	-After processing all invoices, it will print the total amount of all invoices.
<pre>def attendance_report(): print("Generating Attendance Report...") log_data = read_file(user_activity_log_file) # Group logs by date and user attendance_data = {} for log in log_data: username, activity, timestamp = log date = timestamp.split(" ")[0] month_year = "-".join(date.split("-")[:2]) if month_year not in attendance_data: attendance_data[month_year] = {} if username not in attendance_data[month_year]: attendance_data[month_year][username] = timestamp # Display attendance report by month and year for month_year, users in attendance_data.items(): print(f"\nAttendance Report for {month_year}") print("-----") for username, first_login in users.items(): print(f"Employee: {username}, Date: {first_login.split(' ')[0]}, Initial Login Time: {first_login.split(' ')[1]}") print("-----\n") log_activity_file = "log_activity.txt"</pre>	<u>Reading Activity Log</u> - Read user activity logs from file <u>Organize Data</u> - The logs are grouped by month-year and user. The first login time for each user on each day is recorded <u>Generating Report</u> For each month-year, it displays the user's name, date and initial login time
<pre>def log_activity_report(): """ Displays the log of activity report. This function reads the log file and displays the content, which contains all logged activities in the system. """ print("\nDisplaying the Log of Activity Report...\n") try: # Read and display the log entries with open(log_activity_file, "r") as file: log_entries = file.readlines() if log_entries: for entry in log_entries: print(entry.strip()) else: print("No activities logged yet.") except FileNotFoundError: print("Log file not found. No activities have been logged yet.")</pre>	<u>Reading the Log File</u> - Try to open and read the activity log file <u>Display Log Entries</u> - If the file contains entries, it display each activity log If the file is empty or not found, it informs the user that no activities have been logged

<pre>def log_activity(username, activity): """ Logs the activity of a user. Parameters: - username: The username of the user performing the activity. - activity: A description of the activity performed. """ timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S") log_entry = f"{timestamp} - {username} - {activity}\n" # Append the log entry to the log file with open(log_activity_file, "a") as file: file.write(log_entry) print(f"Activity logged: {log_entry.strip()}")</pre>	Create a timestamp to capture the current date and time Formats Log entry by combine the timestamp, username and activity into a log entry Appends to Log file to write the log entry to the log file Display a message to confirm the activity has been logged
<pre>def main(): # Welcome message welcome = """ ----- Welcome to Main Menu ----- print(welcome) while True: print("\n1. Login") print("2. Sign Up") print("3. Exit") choice = input("Enter choice: ").strip() if choice == "1": login() elif choice == "2": sign_up() elif choice == "3": break else: print("Invalid choice!") if __name__ == "__main__": main()</pre>	Show an introduction to the main menu List choices for logging in, signing up or exiting Execute the corresponding function based on user choice “if __name__ == “__main__”;;” is to ensure that “main()” runs when the script is directly executed

Source code	<pre> def manage_inventory(): inventory_header = (""" MANAGE INVENTORY ITEM\n """) greet_user(username) print(inventory_header) while True: print(reminder_msg, "\n(Example: enter '1' to view inventory item.)") print("\nPlease select your action:\n" "1. View Inventory Item\n" "2. Add Inventory Item\n" "3. Remove Inventory Item\n" "4. Update Inventory Item\n" "5. Return to Inventory Main Menu\n") try: choice = int(input("Your input: ")) if choice == 1: view_inventory("inventory_item.txt") elif choice == 2: add_inventory_item("inventory_item.txt", inventory) elif choice == 3: remove_inventory_item("inventory_item.txt", inventory) elif choice == 4: update_inventory_item("inventory_item.txt", inventory) elif choice == 5: break elif choice not in range(1,6): print("Enter selections displayed only.") print("-----\n") except ValueError: print("Enter option NUMBER only.") print("-----\n") </pre>
Explanation	<u>Manage Inventory Function</u> - Creates a menu interface. - Greets user - ‘while True’ loop continuously displays selection menu for next action - Prompts user input (only integer allowed) <u>Error handling</u> - Inform users when input data type is incorrect. <u>Input Validation</u> - Checks if user choice is between valid range.

Source code	<pre> def manage_customer_orders(): CO_header = (""" -----\n """ MANAGE_CUSTOMER_ORDERS\n """ -----\n """) greet_user(username) print(CO_header) while True: print(reminder_msg, "\n(Example: enter '1' to view customer purchase order.)") print("\nPlease select your action:\n" "1. View Customer Purchase Order\n" "2. View Repair/Service Order\n" "3. Repair/Service Information\n" "4. Return to Inventory Main Menu\n") try: choice = int(input("Your input: ")) if choice == 1: CUSpurchase_order() elif choice == 2: repair_service() elif choice == 3: repair_service_information() elif choice == 4: break elif choice not in range(1,5): print("Enter selections displayed only.") print("-----\n") except ValueError: print("Enter option NUMBER only.") print("-----\n") </pre>
Explanation	<u>Manage Customer Orders Function</u> - Creates a menu interface. - Greets user - ‘while True’ loop continuously displays selection menu for next action - Prompts user input (only integer allowed) <u>Error handling</u> - Inform users when input data type is incorrect. <u>Input Validation</u> - Checks if user choice is between valid range.

Source code	<pre> def manage_purchase_orders(): PO_header= ("-----\n" " MANAGE PURCHASE ORDER\n" "-----") greet_user(username) print(PO_header) while True: print(reminder_msg, "\n(Example: enter '1' to generate purchase order.)") print("\nPlease select your action:\n" "1. Generate Purchase Order\n" "2. Check Supplier Purchase Order Status\n" "3. Modify Purchase Order\n" "4. Cancel Supplier Purchase Order\n" "5. View Supplier Purchase Order\n" "6. Return to Inventory Main Menu\n") try: choice = int(input("Your input: ")) if choice == 1: generate_purchase_order("SUP_purchase_orders.txt") elif choice == 2: check_SUPpurchase_order("SUP_purchase_orders.txt") elif choice == 3: modify_SUPpurchase_order("SUP_purchase_orders.txt") elif choice == 4: cancel_SUPpurchase_order("SUP_purchase_orders.txt") elif choice == 5: view_SUPpurchase_order("SUP_purchase_orders.txt") elif choice == 6: break elif choice not in range(1,7): print("Enter selections displayed only.") print("-----\n") except ValueError: print("Enter option NUMBER only.") print("-----\n") </pre>
Explanation	<u>Manage Purchase Orders Function</u> - Creates a menu interface. - Greets user - ‘while True’ loop continuously displays selection menu for next action - Prompts user input (only integer allowed) <u>Error handling</u> - Inform users when input data type is incorrect. <u>Input Validation</u> - Checks if user choice is between valid range.

Source code	<pre> def view_report(): view_report_header= (""" \n """ VIEW REPORTS\n """) greet_user(username) print(view_report_header) while True: print(reminder_msg, "\n(Example: enter '1' to view invoice.)") print("\nPlease select your action:\n" "1. View invoice\n" "2. Inventory Reports\n" "3. Return to Inventory Main Menu\n") try: choice = int(input("Your input: ")) if choice == 1: invoice_report() elif choice == 2: inventory_report() elif choice == 3: break elif choice not in range(1,4): print("Enter selections displayed only.") print(".....\n") except ValueError: print("Enter option NUMBER only.") print(".....\n") </pre>
Explanation	<u>Manage Customer Orders Function</u> - Creates a menu interface. - Greets user - ‘while True’ loop continuously displays selection menu for next action - Prompts user input (only integer allowed) <u>Error handling</u> - Inform users when input data type is incorrect. <u>Input Validation</u> - Checks if user choice is between valid range.

Source code	<pre>def view_inventory(filename): with open(filename, "r") as file: print(file.read())</pre>
Explanation	<u>View Inventory Function</u> - Opens file ("inventory_item.txt") in read mode - Print the content read

Source code	<pre>def add_inventory_item(filename, inventory_list): # Read existing ItemIDs existing_item_ids = set(item[0] for item in inventory_list) # Generate the next ItemID item_id = get_next_item_id(existing_item_ids) sku = get_next_sku(item_id) # Ask for the other required attributes item_name = input("Enter Item Name: ").strip() brand = input("Enter Brand: ").strip() category = input("Enter Category: ").strip() model_number = input("Enter Model Number: ").strip() description = input("Enter Description: ").strip() specification = input("Enter Specification: ").strip() warranty = get_valid_int("Enter Warranty (Year): ") price_per_unit = get_valid_float("Enter Price/Unit (RM): ") quantity_in_stock = get_valid_int("Enter Quantity In Stock: ") # Create a new item list new_item = [item_id, item_name, brand, category, model_number, description, specification, warranty, sku, price_per_unit, quantity_in_stock] # Append the new item to the in-memory list inventory_list.append(new_item) # Write the new item to the file (Append mode) with open(filename, "a") as file: file.write(", ".join(map(str, new_item)) + "\n") print(f"Item added successfully with Item ID: {item_id} and SKU: {sku}")</pre>
Explanation	<u>Add Inventory Item Function Function</u> - Read existing item_id - Calls for <i>get_next_item_id</i> function to get the item_id for the new item. - Calls for <i>get_next_sku</i> function to get the sku for the new item. <u>Accept User Input and Data Validation</u> - Strip leading/trailing space when accepting input. - Use <i>get_valid_int</i> and <i>get_valid_float</i> function to validate integer and float input. <u>Appending text file</u> - Store user input in new list before appending the line in file

Source code	<pre> def remove_inventory_item(filename, inventory_list): item_id = input("Enter the Item ID of the item you want to remove: ").strip() # Find the item in the inventory item_to_remove = None for item in inventory_list: if item[0] == item_id: item_to_remove = item break if item_to_remove: print(f"Item found: {item_to_remove}") while True: confirmation = input(f"Are you sure you want to remove Item ID {item_id}? (yes/no): ").strip().lower() if confirmation in ['yes', 'no']: break else: print("Invalid input. Please enter 'yes' or 'no'.") if confirmation == 'yes': inventory_list.remove(item_to_remove) # Rewrite the entire inventory to the file with open(filename, "w") as file: for item in inventory_list: file.write(", ".join(map(str, item)) + "\n") print(f"Item with ID {item_id} has been removed successfully.") else: print("No changes were made.") else: print(f"Item with ID {item_id} not found in the inventory.") </pre>
Explanation	<u>Remove Inventory Item Function</u> - Initiates by asking for user input (orderId). - Search for the ordered input in the inventory list. - If orderId not found, will inform user. - If orderId found, print the information of the selected item. - Ask for user confirmation before removing file. - If user confirmation is anything other than 'yes', item will not be removed.

Source code	<pre> def update_inventory_item(filename, inventory_list): item_id = input("Enter the Item ID of the item you want to modify: ").strip() # Search for the item in the inventory item_to_update = None for item in inventory_list: if item[0] == item_id: item_to_update = item break if item_to_update: print(f"Item found: {item_to_update}") # List out the attributes for the user to choose from (excluding Item ID and SKU) attributes = ["Item Name", "Brand", "Category", "Model Number", "Description", "Specification", "Warranty (Years)", "Price/Unit (RM)", "Quantity In Stock"] print("Which attribute do you want to update?") for idx, attribute in enumerate(attributes, start=1): print(f"{idx}. {attribute}") # Get the user's choice choice = get_valid_int("Enter the number corresponding to the attribute: ") if 1 <= choice <= len(attributes): attribute_name = attributes[choice - 1] # Mapping user's choice to the correct index in item_to_update mapping = [1, 2, 3, 4, 5, 6, 7, 9, 10] # Corrected mapping correct_index = mapping[choice - 1] current_value = item_to_update[correct_index] print(f"Current {attribute_name}: {current_value}") # Get new value from the user if choice in [7, 9]: # If updating Warranty or Quantity In Stock (integer fields) new_value = get_valid_int(f"Enter new {attribute_name}: ") elif choice == 8: # If updating Price/Unit (float field) new_value = get_valid_float(f"Enter new {attribute_name}: ") else: # For all other fields (strings) new_value = input(f"Enter new {attribute_name}: ").strip() # Ask for confirmation confirmation = input(f"Are you sure you want to update {attribute_name} to '{new_value}'? (yes/no): ").strip().lower() if confirmation == 'yes': item_to_update[correct_index] = str(new_value) # Rewrite the entire inventory to the file with open(filename, "w") as file: for item in inventory_list: file.write(" ".join(map(str, item)) + "\n") print(f"{attribute_name} has been updated successfully for Item ID {item_id}.") else: print("No changes were made.") else: print("Invalid choice. No changes were made.") else: print(f"Item with ID {item_id} not found in the inventory.") </pre>
Explanation	<u>Update Inventory Item Function</u> - Start by asking for item_id from user. - Search inventory list for matching item_id. - If not found, inform user. - If found, set it to item_to_update. - Display the details of the selected item. - Allow users to select the attribute to update by entering corresponding number. - If user choice is not within selection provided, inform user. - If user choice does not follow the correct data type, inform user. - Users are not allowed to update item_id and sku to maintain data integrity. - Mapping is used to make sure updates are stored in the correct index. - Allow user to enter a new value for the selected attribute. - Validate data types when accepting integer and float input. - Write changes to file.

Source code

```
def repair_service_information():
    service_options = display_service_options()
    repair_options = display_repair_options()

    while True:
        print("\nService/Repair Information Menu")
        print("1. Service")
        print("2. Repair")
        print("x. Exit")
        choice = input("Choose an option (1 for Service, 2 for Repair, x to Exit): ")

        if choice == 'x':
            print("Exiting menu.")
            break

        if choice == '1':
            while True:
                print("Available Service Options:")
                for key, value in service_options.items():
                    print(f"{key}. {value['name']}")
                print("r. Return to previous menu")
                print("x. Exit")
                service_choice = input("Select a service option (1-4), 'r' to return, or 'x' to exit: ")

                if service_choice == 'x':
                    print("Exiting menu.")
                    return
                elif service_choice == 'r':
                    break

                if service_choice in service_options:
                    while True:
                        print(f"\n--- {service_options[service_choice]['name']} ---")
                        display_options_with_exit_and_return(service_options[service_choice]['options'])
                        specific_choice = input("Select 'r' to return, or 'x' to exit: ")

                        if specific_choice == 'x':
                            print("Exiting menu.")
                            return
                        elif specific_choice == 'r':
                            break

                        else:
                            print("Invalid choice. Returning to previous menu.")
                    else:
                        print("Invalid choice. Returning to previous menu.")

            elif choice == '2':
                while True:
                    print("Available Repair Options:")
                    for key, value in repair_options.items():
                        print(f"{key}. {value['name']}")
                    print("r. Return to previous menu")
                    print("x. Exit")
                    repair_choice = input("Select a repair option (1-2), 'r' to return, or 'x' to exit: ")

                    if repair_choice == 'x':
                        print("Exiting menu.")
                        return
                    elif repair_choice == 'r':
                        break

                    if repair_choice in repair_options:
                        while True:
                            print(f"\n--- {repair_options[repair_choice]['name']} ---")
                            display_options_with_exit_and_return(repair_options[repair_choice]['options'])
                            specific_choice = input("Select 'r' to return, or 'x' to exit: ")

                            if specific_choice == 'x':
                                print("Exiting menu.")
                                return
                            elif specific_choice == 'r':
                                break

                            else:
                                print("Invalid choice. Returning to previous menu.")
                        else:
                            print("Invalid choice. Returning to previous menu.")

                    else:
                        print("Invalid choice. Returning to previous menu.")

            else:
                print("Invalid choice. Returning to previous menu.")
```

Explanation	<u>Repair/Service Information Function</u> - Start by displaying first layer of submenu. (service/repair) - Allow user to choose which second layer of submenu to go. (broader category) - (by enter corresponding selection number) - Once arrived the second submenu, users can choose which third submenu to go. - (listing of service/repair names and prices) - When user input invalid data type, function is immediately halted. - Users are given the choice to return to previous page/ exit the function.
-------------	---

Source code	<pre> def view_SUPpurchase_order(file_path): # Read the orders from the file orders = read_file(file_path) # Ask the user to input an Order ID order_id = input("Enter the Order ID: ").strip() # Search for the specified order ID for order in orders: if order[0] == order_id: # If the order is found, display its details print(f"\nOrder ID: {order[0]}") print(f"Item ID: {order[2]}") print(f"Delivery Status: {order[3]}") print(f"Payment Status: {order[4]}") print(f"Order Date: {order[8]}") print(f"Order Time: {order[9]}") print(f"Quantity Ordered: {order[5]}") print(f"Unit Price: {order[6]}") print(f"Price Per Item: {order[7]}") print(f"Total Price: {order[10]}") print(f"Expected Delivery Date: {order[11]}") print(f"Remarks: {order[12]}") return # If the order ID is not found, print a message print("Order ID not found.") </pre>
Explanation	<u>View Supplier Purchase Order Function</u> - Start by reading supplier purchase order from the text file. - Prompt for user input. - Validate input. - If orderID not found, inform user. - If found, display all details of the selected order.

Source code	<pre> def generate_purchase_order(file_path): # Load existing orders from file existing_orders = read_file(file_path) # Get next Order ID order_ids = [order[0] for order in existing_orders] order_id = get_next_order_id(order_ids) # Initialize the order order = [order_id, "1"] # Only one item per order # Initialize total price total_price = 0 # Get item details while True: item_id = input("Enter Item ID: ").strip() if any(item[0] == item_id for item in inventory): break else: print("Invalid input. Item ID not found in inventory.") quantity_ordered = int(input("Enter Quantity Ordered: ")) # Find the item in inventory and get its unit price item = next(item for item in inventory if item[0] == item_id) unit_price = item[9] price_per_item = unit_price * quantity_ordered total_price += price_per_item # Append item details to the order order.extend([item_id, "Pending", "Unpaid", str(quantity_ordered), str(unit_price), str(price_per_item)]) # Get the current date and time now = datetime.datetime.now() order_date = now.strftime("%Y-%m-%d") order_time = now.strftime("%H:%M:%S") # Calculate expected delivery date (30 days after order date) expected_delivery_date = (now + datetime.timedelta(days=30)).strftime("%Y-%m-%d") # Add the remaining order details order.extend([order_date, order_time, str(total_price), expected_delivery_date]) # Get remark from the user remark = input("Enter any remarks: ").strip() order.append(remark) # Write the order to the file append_to_file(file_path, order) return order </pre>
Explanation	<u>Generate Purchase Order Function</u> - Start by loading existing purchase order from text file - Use <i>get_next_order_id</i> to generate order for new order. - Ask for a valid item_id and ask for quantity of the order. - Find the unit price from inventory list and calculate the other prices. - Delivery status and Payment status have default values. - Order date and time are automatically recorded. - Expected delivery dates are current date + 30days - After all values are recorded, they are then appended to file.

Source code	<pre> def check_SUPpurchase_order(file_path): supplier_order_data = read_file(file_path) print("Purchase Order (SUPPLIER)") # Display all supplier orders and their statuses for order in supplier_order_data: print(f"Order ID: {order[0]}, Payment Status: {order[4]}, Delivery Status: {order[3]}") # Check status of a specific supplier order print("\n**Example Order ID: order001 \nEnter Order ID to check status: ") order_id = input().strip() # Search for the specified order ID for order in supplier_order_data: if order[0] == order_id: print(f"Order ID: {order[0]}, Payment Status: {order[4]}, Delivery Status: {order[3]}") break else: print("Order ID not found.") </pre>
Explanation	<u>Check Supplier Purchase Order Function</u> - Start by loading supplier purchase order. - Print all statuses of all orderIDs of available supplier purchase order. - To check the status of a specific supplier purchase order, simply input the orderID.

Source code	<pre> def cancel_SUPpurchase_order(file_path): # Read the orders from the file orders = read_file(file_path) # Ask the user to input an Order ID to cancel order_id = input("Enter the Order ID to cancel: ").strip() # Search for the specified order ID for order in orders: if order[0] == order_id: # If the order is found, display its details print(f"\nOrder ID: {order[0]}") print(f"Item ID: {order[2]}") print(f"Delivery Status: {order[3]}") print(f"Payment Status: {order[4]}") print(f"Order Date: {order[8]}") print(f"Order Time: {order[9]}") print(f"Quantity Ordered: {order[5]}") print(f"Total Price: {order[10]}") print(f"Expected Delivery Date: {order[11]}") print(f"Remarks: {order[12]}") # Confirm if the user wants to cancel the order confirm = input("Do you want to cancel this order? (yes/no): ").strip().lower() if confirm == "yes": # Remove the order from the list orders.remove(order) # Update the file with the remaining orders write_file(file_path, orders) print(f"Order {order_id} has been canceled.") else: print("Order cancellation aborted.") return # If the order ID is not found, print a message print("Order ID not found.") </pre>
Explanation	<u>Cancel Supplier Purchase Order Function</u> - Start by loading the supplier purchase orders. - Prompt for user input (orderId). - If not found, inform user. - If found, display all related information. - Ask for user confirmation. - If answer is 'yes', the order will be removed. - Any input other than that will abort the cancellation.

Source code

```
def modify_SUPurchase_order(file_path):
    # Read the orders from the file
    orders = read_file(file_path)

    # Ask the user to input an Order ID to modify
    order_id = input("Enter the Order ID to modify: ").strip()

    # Search for the specified order ID
    for order in orders:
        if order[0] == order_id:
            # If the order is found, display its current details
            print(f"\nOrder ID: {order[0]}")
            print(f"Item ID: {order[2]}")
            print(f"Delivery Status: {order[3]}")
            print(f"Payment Status: {order[4]}")
            print(f"Order Date: {order[8]}")
            print(f"Order Time: {order[9]}")
            print(f"Quantity Ordered: {order[5]}")
            print(f"Unit Price: {order[6]}")
            print(f"Price Per Item: {order[7]}")
            print(f"Total Price: {order[10]}")
            print(f"Expected Delivery Date: {order[11]}")
            print(f"Remarks: {order[12]}")

            # Ask the user what attribute they want to modify
            attribute_to_modify = input("\nWhich attribute do you want to modify? (Item ID, Delivery Status, Payment Status, Quantity Ordered, Remarks): ").strip().lower()

            # Validate input and modify the attribute
            if attribute_to_modify == "item id":
                new_item_id = input("Enter new Item ID: ").strip()
                if any(item[0] == new_item_id for item in inventory):
                    order[2] = new_item_id
                    # Update the unit price based on the new item ID
                    item = next(item for item in inventory if item[0] == new_item_id)
                    unit_price = float(item[6])
                    order[6] = str(unit_price) # Update unit price

                    # Recalculate the price per item and total price based on the current quantity
                    quantity_ordered = int(order[5])
                    price_per_item = unit_price * quantity_ordered
                    order[7] = str(price_per_item) # Update price per item
                    order[10] = str(price_per_item) # Update total price (single item order)
                else:
                    print("Invalid Item ID. Modification aborted.")
                    return
            elif attribute_to_modify == "quantity ordered":
                new_quantity_ordered = input("Enter new Quantity Ordered: ").strip()
                if new_quantity_ordered.isdigit() and int(new_quantity_ordered) > 0:
                    order[5] = new_quantity_ordered
                    # Recalculate the price per item and total price based on the new quantity
                    unit_price = float(order[6])
                    price_per_item = unit_price * int(new_quantity_ordered)
                    order[7] = str(price_per_item) # Update price per item
                    order[10] = str(price_per_item) # Update total price (single item order)
                else:
                    print("Invalid Quantity. Modification aborted.")
                    return
            # Other modifications (Delivery Status, Payment Status, Remarks) don't affect pricing
            elif attribute_to_modify == "delivery status":
                new_delivery_status = input("Enter new Delivery Status (pending/shipping/delivered): ").strip().lower()
                if new_delivery_status in ["pending", "shipping", "delivered"]:
                    order[3] = new_delivery_status
                else:
                    print("Invalid Delivery Status. Modification aborted.")
                    return
            elif attribute_to_modify == "payment status":
                new_payment_status = input("Enter new Payment Status (paid/unpaid): ").strip().lower()
                if new_payment_status in ["paid", "unpaid"]:
                    order[4] = new_payment_status
                else:
                    print("Invalid Payment Status. Modification aborted.")
                    return
            elif attribute_to_modify == "remarks":
                new_remarks = input("Enter new Remarks: ").strip()
                order[12] = new_remarks
            else:
                print("Invalid attribute. Modification aborted.")
                return

            # Recalculate the Expected Delivery Date based on logic (e.g., add 7 days to current date)
            now = datetime.datetime.now()
            order[8] = now.strftime("%Y-%m-%d") # Update order date
            order[9] = now.strftime("%H:%M:%S") # Update order time
            expected_delivery_date = now + datetime.timedelta(days=7) # Add 7 days for delivery
            order[11] = expected_delivery_date.strftime("%Y-%m-%d")

            # Update the file with the modified order
            write_file(file_path, orders)
            print(f"Order {order_id} has been modified.")
            return

    # If the order ID is not found, print a message
    print("Order ID not found.")
```

Explanation

Modify Supplier Purchase Order Function

- Start by reading all existing orders.
- Prompt for orderID.
- Search for matching ID in loaded file.
- If not found, inform user.
- If found, ask which attribute to edit by entering the corresponding name in selection provided.
- If item_id changed, the corresponding details (mainly prices) will be found, recalculated and updated.
- System will validate data type for input such as quantity.
- For statuses, only selections provided are valid for input.
- The dates and times will be automatically adjusted to the time of modification.
- After modification is done, append update to file and inform user.

Source code	<pre> def inventory_report(): # Load inventory list from the inventory file inventory_list = load_inventory("inventory_item.txt") # Create an empty list for the inventory report inventory_report_list = [] # Iterate through each item in the inventory and calculate total value for item in inventory_list: quantity_in_stock = int(item[-1]) unit_price = float(item[-2]) # Calculate total value total_value = quantity_in_stock * unit_price # Add total value as a new column to the item data item.append(f"{total_value:.2f}") # Add the updated item to the inventory report list inventory_report_list.append(item) # Write the inventory report list to the "inventory_report.txt" file with open("inventory_report.txt", "w") as file: for item in inventory_report_list: file.write(", ".join(item) + "\n") # Read and print the final result from the "inventory_report.txt" file print("\nFinal Inventory Report:") with open("inventory_report.txt", "r") as file: print(file.read()) </pre>
Explanation	<u>View Inventory Report Function</u> - Start by loading the inventory list. - For each item in the list, calculate their respective total values and add them as a new column in the new list. - Append all the lists into the new list (inventory_report_list). - Write the new list into a new file and print the content.

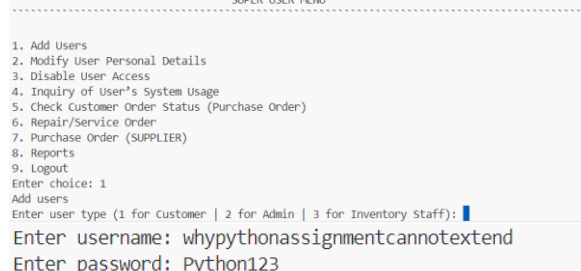
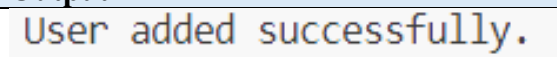
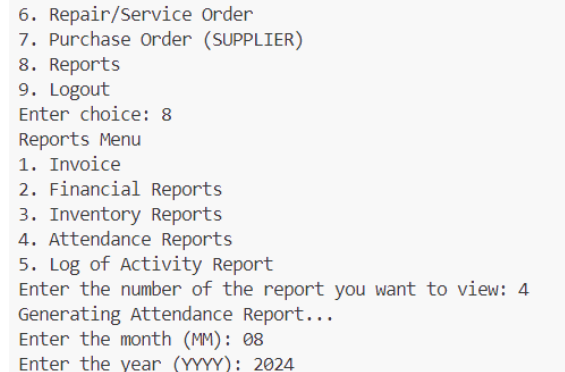
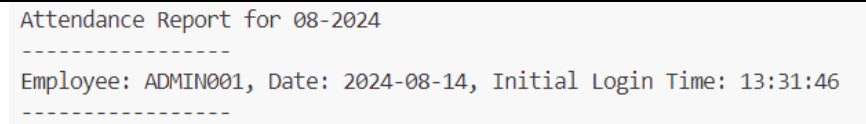
Additional features source code

Source code	Explanation
<pre>def get_next_admin_username(login_data): # Find the highest existing admin username admin_usernames = [user[0] for user in login_data if user[3] == "2" and user[0].startswith("ADMIN")] if not admin_usernames: return "ADMIN001" # Extract numbers from existing admin usernames, find the highest, and increment it max_number = max(int(username[5:]) for username in admin_usernames) next_number = max_number + 1 return f"ADMIN{next_number:03d}"</pre>	<u>Get Next Admin Username Function</u> To generate the next available admin's username. In sign up function, it will generate a username for admin.
<pre>if user_type == "1": if not (len(password) >= 8 and any(c.isupper() for c in password) and sum(c.isdigit() for c in password) >= 3): print("Customer password must be at least 8 characters long, including 1 UPPER CASE and 3 NUMBERS.") return elif user_type == "2": if not (len(password) >= 10 and password.startswith("ADMIN@")): print("Admin password must be at least 10 characters long and start with 'ADMIN@'.") return elif user_type == "3": if not (len(password) >= 8 and password.startswith("INV@") and sum(c.isdigit() for c in password) == 4): print("Inventory staff password must be at least 8 characters long, start with 'INV@' and followed by 4 numbers.") return</pre>	<u>Password Validation</u> To handle password validation for customers, admin and inventory staff. - Customer password must be at least 8 characters long, including 1 upper case and 3 numbers. - Admin password must be at least 10 characters long and start with "ADMIN@". - Inventory staff password must be at least 8 characters long and start with "INV@" and followed by numbers.

<pre>def generate_order_id(): counter_file = 'order_id_counter.txt' if not os.path.isfile(counter_file): with open(counter_file, 'w') as f: f.write('10000000') with open(counter_file, 'r') as f: current_value = int(f.read().strip()) new_value = current_value + 1 with open(counter_file, 'w') as f: f.write(str(new_value)) order_id = f"{new_value:08d}" return order_id</pre>	This function creates a unique 8-digit order ID by reading, incrementing, and updating a counter stored in `order_id_counter.txt`. If the file doesn't exist, it starts the counter at `10000000`. The updated value is saved back to the file and returned as the new order ID.
<pre># Group logs by date and user attendance_data = {} for log in log_data: username, activity, timestamp = log date = timestamp.split(" ")[0] month_year = "-".join(date.split("-")[:2]) if month_year not in attendance_data: attendance_data[month_year] = {} if username not in attendance_data[month_year]: attendance_data[month_year][username] = timestamp # Display attendance report by month and year for month_year, users in attendance_data.items(): print(f"\nAttendance Report for {month_year}") print("-----") for username, first_login in users.items(): print(f"Employee: {username}, Date: {first_login.split(' ')[0]}, Initial Login Time: {first_login.split(' ')[1]}") print("-----\n")</pre>	In attendance reports function, superuser must enter the month and year they wanted to check for attendance reports.
<pre>def get_next_order_id(existing_order_ids): if not existing_order_ids: return "order001" # Start with "order001" if no orders exist # Convert order IDs to integers and find the highest one max_id = max(int(order_id[5:]) for order_id in existing_order_ids) return f"order{max_id + 1:03d}"</pre>	Automatically generate a new orderID for the new supplier purchase order.
<pre>def get_next_item_id(existing_item_ids): if not existing_item_ids: return "101" # Start with "101" if no items exist # Convert item IDs to integers and find the highest one max_id = max(int(item_id) for item_id in existing_item_ids) return str(max_id + 1)</pre>	Automatically generate a new item_id for the new item in inventory list.
<pre>def get_next_sku(item_id): return f"SKU{item_id}"</pre>	Automatically generate a new sku for the new item in inventory list.

<pre>def get_valid_int(prompt): while True: try: value = int(input(prompt).strip()) return value except ValueError: print("Incorrect data type, please enter an integer.")</pre>	Validate data type for input that accept integer input only.
<pre>def get_valid_float(prompt): while True: try: value = float(input(prompt).strip()) return value except ValueError: print("Incorrect data type, please enter a number.")</pre>	Validate data type for input that accept float input only.

Screenshots of sample input/output with explanation

Input	Explanation
 <pre> 1. Add Users 2. Modify User Personal Details 3. Disable User Access 4. Inquiry of User's System Usage 5. Check Customer Order Status (Purchase Order) 6. Repair/Service Order 7. Purchase Order (SUPPLIER) 8. Reports 9. Logout Enter choice: 1 Add users Enter user type (1 for Customer 2 for Admin 3 for Inventory Staff): Enter username: whypythonassignmentcannotextend Enter password: Python123 </pre>	Superuser need to input user type, username(username for admin is auto generated) and password.
Output	Explanation
 <pre> User added successfully. </pre>	It will display “User added successfully” message once superuser fill in the correct username and password (based on the
Input	Explanation
 <pre> 6. Repair/Service Order 7. Purchase Order (SUPPLIER) 8. Reports 9. Logout Enter choice: 8 Reports Menu 1. Invoice 2. Financial Reports 3. Inventory Reports 4. Attendance Reports 5. Log of Activity Report Enter the number of the report you want to view: 4 Generating Attendance Report... Enter the month (MM): 08 Enter the year (YYYY): 2024 </pre>	Superuser enter “8” for reports function, and then enter “4” in reports menu for “Attendance Reports”. After that, superuser need to enter the month and year for the attendance report superuser would like to check.
Output	Explanation
 <pre> Attendance Report for 08-2024 ----- Employee: ADMIN001, Date: 2024-08-14, Initial Login Time: 13:31:46 ----- </pre>	It will display Attendance Report based on the month and year given by superuser.
Input	Explanation

<pre> ----- Admin Menu ----- 1. Verify New Customer 2. Check Customer Order Status (Purchase Order) 3. Repair/Service Order 4. Purchase order (SUPPLIER) 5. Reports 6. Logout Enter choice: 1 Verify New Customer Pending Customers: Username: serenenotserena Enter the username of the customer to approve: serenenotserena </pre>	Admin entered choice “1” to verify customer. After that, it will display customer in pending list and prompt for username to verify.
Output	Explanation
<pre> Customer 'serenenotserena' has been approved. </pre>	If the customer status is pending, it will be approved after admin enter the username.
Input	Explanation
<pre> ----- Welcome to Customer Menu ----- 1. Purchase Order 2. Repair/Service Order 3. Modify Purchase Order/Repair/Service Order 4. Payment for orders placed 5. Inquiry of order status 6. Cancel Order 7. Reports 8. Logout Enter choice: 2 Service/Repair Order Menu 1. Service 2. Repair Choose an option (1 for Service, 2 for Repair): 1 Available Service Options: 1. Hardware Services 2. Software Services 3. IT Services and Support 4. Networking Services Select a service option (1-4): 4 --- Networking Services --- a. Wi-Fi/LAN networking setup: RM100 b. Wi-Fi/LAN networking upgrades: RM100 c. Wireless setup or installation: RM150 d. Extend internet/Wi-Fi coverage: RM300 Select a specific service option (a-h): c Please select your Device Transfer Method: 1. Drop off 2. Pickup (additional RM30) Enter your choice (1 or 2): 1 Business hours: Mon-Fri 9AM - 6PM Total Amount (including additional charges): RM150.00 Are you sure you want to place the order? (yes/no): yes </pre>	Customer input “2” for Repair/Service Order on Customer Menu. Then input “1” for service. In service page, customer input “4” for Networking Services and then choose the service customer wants. Customer must choose their device transfer method, and then select pick up time (for pick up selection). Lastly, it will display the amount and confirmation message.
Output	Explanation
<pre> Order placed successfully! Your order ID is 10000007. Please proceed to make payment. </pre>	After the customer enters “yes”, it will display “Order placed successfully” message, order ID,

	and reminder message for payment.
Input	Explanation
<pre> Welcome,inv_Felix Diu ----- INVENTORY MAIN MENU ----- **Select action by entering option number only. (Example: enter '1' to manage inventory item.) Please select your action: 1. Manage Inventory Item 2. Manage Customer Orders 3. Manage Purchase Orders 4. View Report 5. Logout Welcome,inv_Felix Diu ----- MANAGE INVENTORY ITEM ----- **Select action by entering option number only. (Example: enter '1' to view inventory item.) Please select your action: 1. View Inventory Item 2. Add Inventory Item 3. Remove Inventory Item 4. Update Inventory Item 5. Return to Inventory Main Menu Your input: 4 Enter the Item ID of the item you want to modify: 118 Item found: ['118', 'USB Flash Drive', 'SanDisk', 'Storage', 'Ultra Flair', 'High-speed USB flash drive', '128GB', 'USB 3.0', '1', 'SKU118', '65.0', '75'] Which attribute do you want to update? 1. Item Name 2. Brand 3. Category 4. Model Number 5. Description 6. Specification 7. Warranty (Years) 8. Price/unit (RM) 9. Quantity In Stock Enter the number corresponding to the attribute: 9 Current Quantity In Stock: 65.0 Enter new Quantity In Stock: 80 Are you sure you want to update Quantity In Stock to '80'? (yes/no): yes </pre>	Inventory staff enter “1” to manage inventory item. In inventory item menu, inventory staff enter “4” to update inventory item. After that, inventory staff enter “9”. After that, inventory staff enter the item id he/she wanted to update. Inventory staff enter “9” to modify the quantity, the quantity change from 65 to 80.
Output	Explanation
<pre> Quantity In Stock has been updated successfully for Item ID 118. </pre>	After the inventory staff enter “yes”, it display successfully updated message.

Conclusion

The system for KLCCC will enhance operations by offering specialized functions for Customer, Admins, Inventory Staffs and even Superuser. It will manage registrations, handle multiple orders, process payments and generate reports based on the user's user type. It aims to enhance efficiency, optimize transactions and support the company's growth.

Task Division

Name	TP number	Task	Signature
Serene Loh Zi Ting	TP075920	Sign up Login Main menu Admin function Superuser function - Pseudocode - Flowchart - Source code - conclusion	
Siew Zhen Lynn	TP076386	Functionalities of Customer management - Pseudocode - Flowchart - Source code - Introduction	celine.
Yong Xuan Lyn	TP076797	Functions for Inventory Management: - Source code - Pseudocode - Flowchart - Additional functions	